

Java 27 — Prepare to Make final Mean Final

Java Version: 27

Year: 2026

Feature: Prepare to Make final Mean Final

UniqueClassName: FinalMeansFinal

Note: Java 27 is targeted for **2027**, so this feature is still future-facing as of October 2026. The idea is to strengthen the guarantee around final fields so that code cannot use deep reflection or other special mechanisms to change them after initialization. Current Java specifications still describe exceptional mechanisms that can modify some final fields after construction. [OpenJDK](#)

1. Definition

Prepare to Make final Mean Final is an effort to strengthen Java's integrity model so that a field declared final truly behaves as immutable after its permitted initialization.

Normally:

```
final int value = 100;
```

means that value is assigned once and cannot subsequently be reassigned through ordinary Java code.

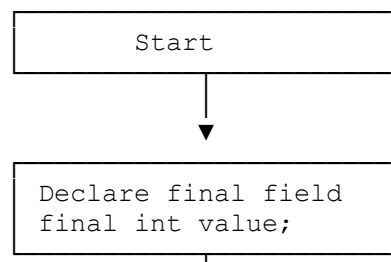
Historically, however, certain mechanisms—particularly **deep reflection** and implementation-dependent techniques—could modify some final instance fields after construction. The Java Language Specification explicitly recognizes such exceptional modification today. [OpenJDK](#)

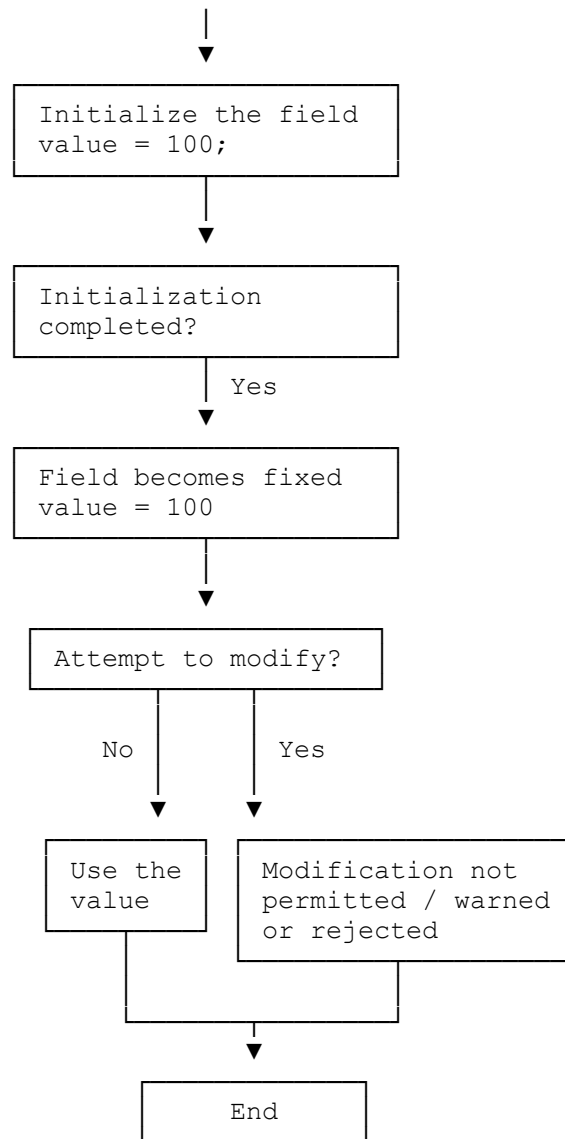
The direction of this feature is therefore:

final field → initialize once → no later mutation, including through mechanisms that previously bypassed normal Java-language restrictions.

This gives the JVM stronger guarantees for **immutability, optimization, security, reliability, and reasoning about objects**. Java already gives final fields special Java Memory Model semantics that help safely construct immutable objects. [OpenJDK](#)

2. Conceptual Flowchart





3. Java Code Example

```
public class FinalMeansFinal {  
  
    private final String employeeId;  
    private final String employeeName;  
  
    public FinalMeansFinal(String employeeId, String employeeName) {  
        // Legal initialization of final fields  
        this.employeeId = employeeId;  
        this.employeeName = employeeName;  
    }  
  
    public String getEmployeeId() {  
        return employeeId;  
    }  
}
```

```

public String getEmployeeName() {
    return employeeName;
}

public static void main(String[] args) {

    FinalMeansFinal employee =
        new FinalMeansFinal("EMP101", "Anurag Bharti");

    System.out.println("Employee ID    : "
        + employee.getEmployeeId());

    System.out.println("Employee Name : "
        + employee.getEmployeeName());

    // Not allowed:
    //
    // employee.employeeId = "EMP999";
    //
    // Compile-time error:
    // Cannot assign a value to final variable employeeId
}
}

```

Output:

```

Employee ID    : EMP101
Employee Name  : Anurag Bharti

```

4. What final Means

Usage	Meaning
final local variable	Can be assigned only once
final field	Reference/value cannot normally be reassigned after initialization
final method	Cannot be overridden by a subclass
final class	Cannot be extended
final object reference	Reference cannot change, but the referenced object's internal state may still be mutable
Final Means Final direction	Strengthens protection against mechanisms that circumvent normal final restrictions

For example, this distinction remains important:

```
final StringBuilder builder = new StringBuilder("Java");

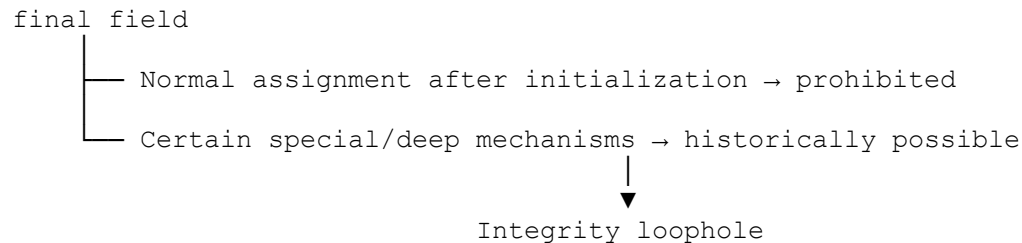
// Allowed: object itself is mutable
builder.append(" 27");

// Not allowed: changing the reference
// builder = new StringBuilder("New");
```

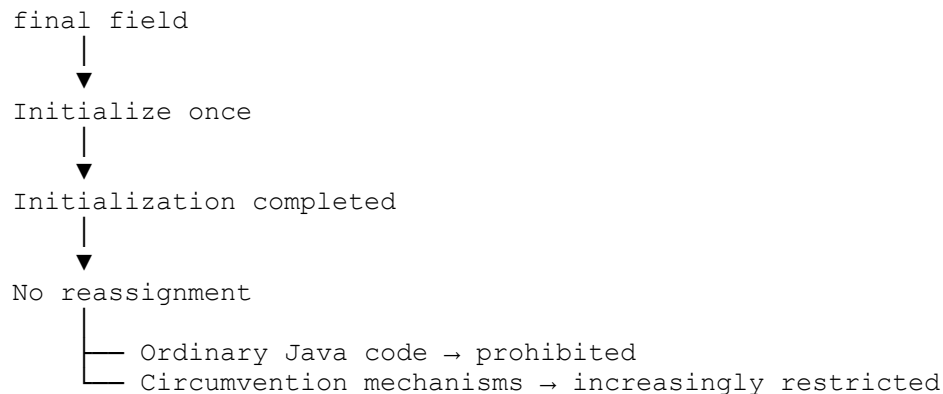
So “**final means final**” does not automatically mean that every object referenced by a final variable becomes deeply immutable. The Java specification says a final reference always refers to the same object, while that object's state may still change. [OpenJDK](#)

5. Core Idea

Before stronger enforcement



Stronger "final means final" model



In short, the goal is to make the programmer's declaration:

```
private final int value;
```

carry a stronger runtime meaning: **once legitimately initialized, the field stays final**, rather than final being a restriction that privileged mechanisms can routinely bypass.

Java 27 — Post-Quantum Hybrid Key Exchange for TLS 1.3

Java Version: 27

Year: 2026

Feature: Post-Quantum Hybrid Key Exchange for TLS 1.3

UniqueClassName: PostQuantumHybridKeyExchangeTLS13

Important: As of October 2026, this should be treated as a **future/target capability**, not a confirmed standard Java 27 API feature with a guaranteed class or method named PostQuantumHybridKeyExchangeTLS13. Current OpenJDK APIs provide TLS 1.3, named-group configuration, KEM support, and key-agreement APIs, but the exact hybrid TLS 1.3 integration depends on the JDK/provider and the final Java 27 specifications. [OpenJDK](#)

1. Definition

Post-Quantum Hybrid Key Exchange for TLS 1.3 combines a traditional key-exchange mechanism with a **post-quantum cryptographic (PQC)** mechanism during a TLS 1.3 handshake.

The conceptual goal is:

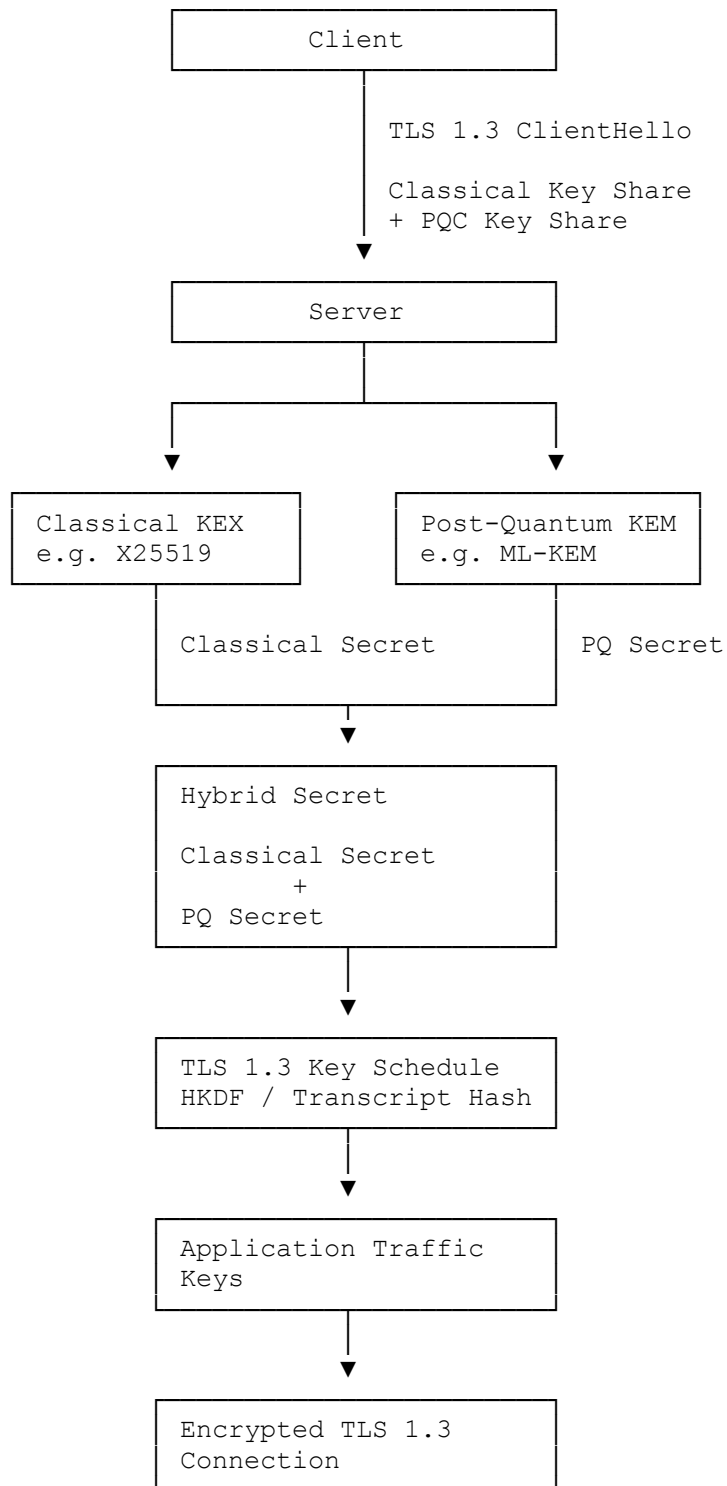
```
Classical security
      +
Post-quantum security
      ↓
Hybrid shared secret
      ↓
TLS 1.3 key derivation
      ↓
Encrypted TLS session
```

A hybrid design is intended to provide protection even if one of the component algorithms becomes vulnerable. For example, a deployment might combine a classical elliptic-curve mechanism such as **X25519** with a post-quantum KEM such as **ML-KEM**.

Java already exposes X25519 through KeyAgreement, and Java's cryptographic APIs provide a general KEM abstraction for key encapsulation mechanisms. [OpenJDK](#)

TLS 1.3 itself uses a handshake to establish the cryptographic parameters and session keys before application data is exchanged. [OpenJDK](#)

2. Hybrid Key-Exchange Flowchart



Java's TLS API represents the key-exchange choices through **named groups** in `SSLParameters`; the provider determines which groups are actually supported. [OpenJDK](#)

3. Java Code Example

Because the exact Java 27 hybrid TLS API is not finalized/guaranteed, the following is a **conceptual Java implementation** showing how an application could model the hybrid operation.

```
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.HexFormat;

public class PostQuantumHybridKeyExchangeTLS13 {

    public static void main(String[] args) throws Exception {

        System.out.println("TLS Version: TLS 1.3");
        System.out.println("Key Exchange: Hybrid");
        System.out.println("Classical Component: X25519");
        System.out.println("Post-Quantum Component: ML-KEM");

        // -----
        // Step 1: Simulated classical shared secret
        // -----

        byte[] classicalSecret = new byte[32];
        new SecureRandom().nextBytes(classicalSecret);

        // -----
        // Step 2: Simulated post-quantum shared secret
        // -----

        byte[] postQuantumSecret = new byte[32];
        new SecureRandom().nextBytes(postQuantumSecret);

        // -----
        // Step 3: Combine both secrets
        //
        // A real TLS implementation must use the exact
        // hybrid construction specified by the applicable
        // TLS/PQC standard.
        // -----

        byte[] hybridInput = new byte[
            classicalSecret.length + postQuantumSecret.length
        ];

        System.arraycopy(
            classicalSecret,
            0,
            hybridInput,
            0,
            classicalSecret.length
        );

        System.arraycopy(
```

```

        postQuantumSecret,
        0,
        hybridInput,
        classicalSecret.length,
        postQuantumSecret.length
    );

    // -----
    // Step 4: Derive a hybrid secret
    // -----

    MessageDigest sha256 =
        MessageDigest.getInstance("SHA-256");

    byte[] hybridSecret =
        sha256.digest(hybridInput);

    // -----
    // Step 5: Display demonstration result
    // -----

    System.out.println(
        "Hybrid Secret: "
        + HexFormat.of().formatHex(hybridSecret)
    );

    System.out.println(
        "Hybrid key exchange completed."
    );
}
}

```

Example output

```

TLS Version: TLS 1.3
Key Exchange: Hybrid
Classical Component: X25519
Post-Quantum Component: ML-KEM
Hybrid Secret: 8e4c...f91a
Hybrid key exchange completed.

```

The code above is educational, not a production TLS implementation. In real TLS 1.3, applications should not manually concatenate random values and hash them as a substitute for the protocol's standardized key schedule. The JDK's TLS implementation should perform the actual handshake.

4. Java TLS 1.3 Configuration Concept

Java's SSLParameters allows applications/providers to configure TLS protocols and **key-exchange named groups**. OpenJDK

A conceptual configuration could look like:

```
import javax.net.ssl.SSLContext;
import javax.net.ssl.SSLParameters;

public class TLS13HybridConfiguration {

    public static void main(String[] args) throws Exception {

        SSLContext context =
            SSLContext.getDefault();

        SSLParameters parameters =
            context.getDefaultSSLParameters();

        parameters.setProtocols(
            new String[] {"TLSv1.3"}
        );

        /*
         * Actual hybrid group names depend on the
         * JDK/provider and the finalized standards.
         *
         * Example conceptually:
         *
         * parameters.setNamedGroups(
         *     new String[] {
         *         "X25519MLKEM768",
         *         "X25519"
         *     }
         * );
         */

        System.out.println(
            "TLS 1.3 configured."
        );

        System.out.println(
            "Supported named groups:"
        );

        for (String group :
            parameters.getNamedGroups()) {

            System.out.println("  " + group);
        }
    }
}
```

The important point is that **setNamedGroups()** **does not itself implement a post-quantum algorithm**. It tells the TLS implementation which key-exchange groups it may negotiate; support comes from the underlying security provider/JDK implementation. [OpenJDK](#)

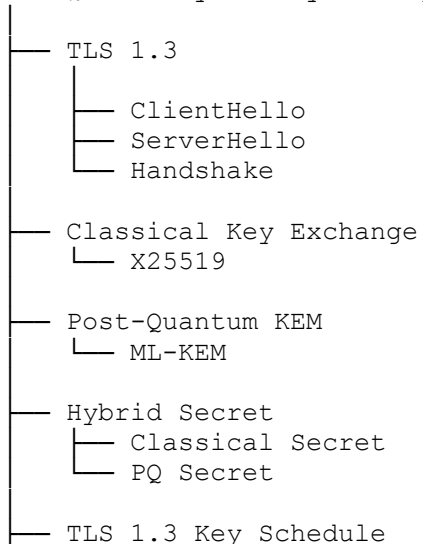
5. Classical vs Post-Quantum vs Hybrid

Approach	Example	Quantum-resistant?	Purpose
Classical	X25519	✗	Current widely deployed key exchange
Post-quantum	ML-KEM	✓	Designed to resist quantum attacks
Hybrid	X25519 + ML-KEM	✓*	Combines classical + PQ protection
TLS encryption	AES-128-GCM / AES-256-GCM	Symmetric	Encrypts application data
TLS key derivation	HKDF	Symmetric	Derives TLS traffic keys

*The security properties depend on the exact standardized hybrid construction and its implementation.

6. Conceptual Java Architecture

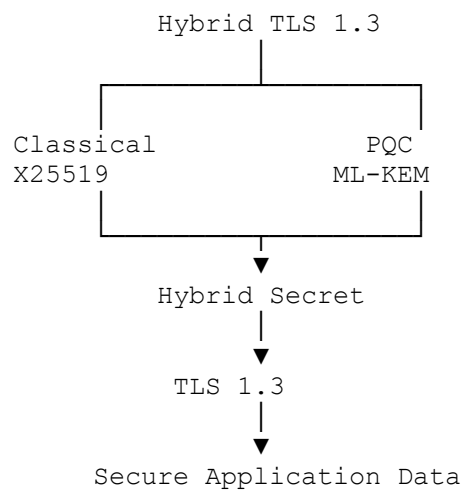
PostQuantumHybridKeyExchangeTLS13





Java's current API direction is compatible with this architecture: KEM provides KEM operations, KeyAgreement provides classical key agreement such as X25519, and SSLParameters provides the TLS named-group configuration layer. [OpenJDK](#)

7. Main Benefit



The fundamental objective is **cryptographic transition**: retain compatibility and security properties associated with classical cryptography while adding protection designed for the post-quantum era.

Java 27 — Make G1 the Default Garbage Collector in All Environments

Java Version: 27

Year: 2027

Feature: Make G1 the Default Garbage Collector in All Environments

UniqueClassName: G1DefaultGarbageCollector

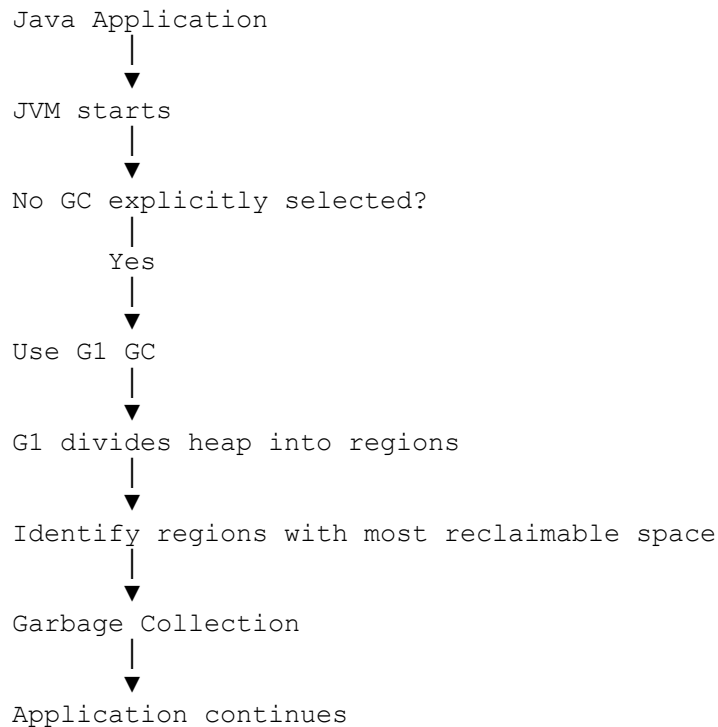
Important: This is a **future-facing Java 27 feature description**. G1 is already the default garbage collector for typical server-class machines in modern JDKs; the proposed direction is to make **G1 the default across environments**, rather than automatically selecting a different collector based on the environment.

1. Definition

G1 (Garbage-First) Garbage Collector is a low-pause, region-based garbage collector designed to provide predictable pause times while efficiently managing large Java heaps.

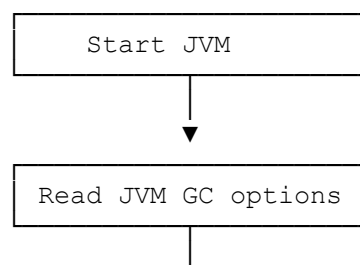
The concept of **making G1 the default garbage collector in all environments** means that a Java application would use **G1 by default**, unless the user explicitly selects another collector through JVM options.

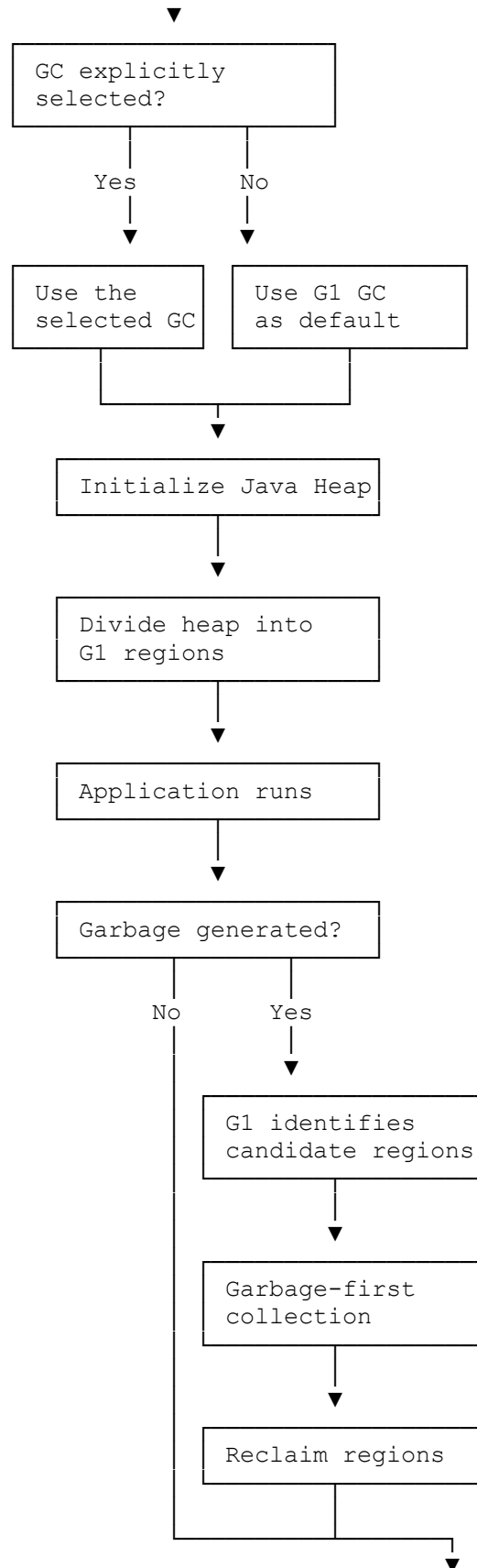
Conceptually:



G1 divides the Java heap into a collection of equal-sized **heap regions** and uses a garbage-first strategy to prioritize regions that provide the most benefit from garbage collection.

2. Flowchart





Continue JVM
execution

3. Java Code Example

The Java program itself does not normally select the garbage collector. The JVM determines the default collector unless a command-line option explicitly selects one.

```
public class G1DefaultGarbageCollector {

    public static void main(String[] args) {

        System.out.println(
            "Java Garbage Collector Information"
        );

        System.out.println("-----");

        System.out.println(
            "Garbage Collector: G1"
        );

        System.out.println(
            "G1 is the default collector in the target configuration."
        );

        System.out.println(
            "Creating temporary objects..."
        );

        for (int i = 0; i < 1_000_000; i++) {

            byte[] data = new byte[1024];

            if (i % 100_000 == 0) {
                System.out.println(
                    "Created objects: " + i
                );
            }

        }

        System.out.println(
            "Application execution completed."
        );

    }

}
```

Example output

```
Garbage Collector Information
-----
Garbage Collector: G1
G1 is the default collector in the target configuration.
Creating temporary objects...
Created objects: 0
Created objects: 100000
Created objects: 200000
...
Application execution completed.
```

4. Verify the Active Garbage Collector

A useful Java example is to query the JVM's garbage-collector MXBeans:

```
import java.lang.management.GarbageCollectorMXBean;
import java.lang.management.ManagementFactory;

public class G1DefaultGarbageCollector {

    public static void main(String[] args) {

        System.out.println("Active Garbage Collectors:");

        for (GarbageCollectorMXBean gc :
            ManagementFactory.getGarbageCollectorMXBeans()) {

            System.out.println(
                "Name: " + gc.getName()
            );

            System.out.println(
                "Collection Count: "
                + gc.getCollectionCount()
            );

            System.out.println(
                "Collection Time: "
                + gc.getCollectionTime()
                + " ms"
            );

            System.out.println();
        }
    }
}
```

With G1, the collector names reported by the JVM typically include names such as:

G1 Young Generation
G1 Old Generation

The exact names can vary by JDK implementation/version.

5. Explicitly Select G1

Even when G1 is the default, you can explicitly request it using:

```
java -XX:+UseG1GC G1DefaultGarbageCollector
```

For example:

```
java -Xms512m -Xmx2g -XX:+UseG1GC G1DefaultGarbageCollector
```

Here:

Option	Purpose
-Xms512m	Initial heap size
-Xmx2g	Maximum heap size
-XX:+UseG1GC	Explicitly select G1
G1DefaultGarbageCollector	Java main class

6. G1 Heap Architecture

G1 does not treat the heap simply as:

Young Generation
+
Old Generation

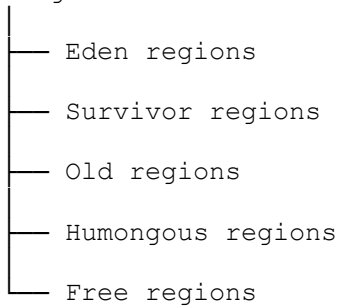
Instead, the heap is divided into many **regions**:

Java Heap

Region	Region	Region	Region	Region
Region	Region	Region	Region	Region
Region	Region	Region	Region	Region
Region	Region	Region	Region	Region

Regions can play different roles during JVM execution:

G1 Heap Regions

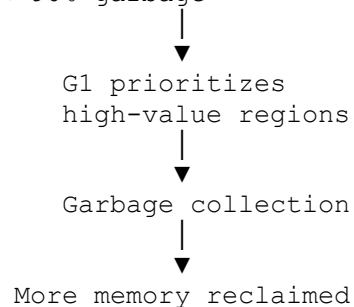


7. Garbage-First Principle

The key idea is to prioritize regions that offer the greatest amount of reclaimable memory relative to the collection cost.

Heap Regions

Region A → 10% garbage
Region B → 40% garbage
Region C → 80% garbage
Region D → 20% garbage
Region E → 90% garbage



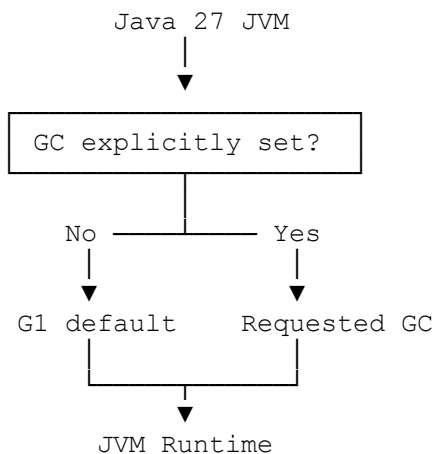
for collection effort

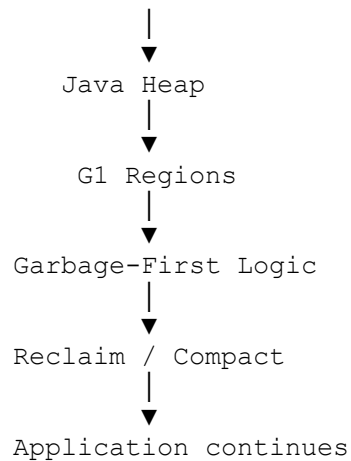
This is where the name **Garbage-First** comes from.

8. Advantages

Advantage	Description
Predictable pauses	Designed to meet pause-time goals
Region-based heap	Heap is divided into manageable regions
Concurrent processing	Several GC activities can occur concurrently with application execution
Large heaps	Designed to work efficiently with large Java heaps
Compaction	Helps reduce fragmentation
Automatic selection	Applications can use G1 without explicitly specifying a collector
Simplified JVM configuration	A common default reduces environment-specific GC selection

9. Conceptual Java 27 Model





Key takeaway

G1DefaultGarbageCollector represents the idea of making **G1 the universal default garbage collector**, reducing the need for the JVM to select different default collectors based on the environment while retaining the ability for users to explicitly select another collector when appropriate.

Java 27 — Lazy Constants (Third Preview)

Java Version: 27

Year: 2026

Feature: Lazy Constants (Third Preview)

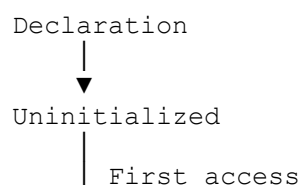
UniqueClassName: LazyConstants

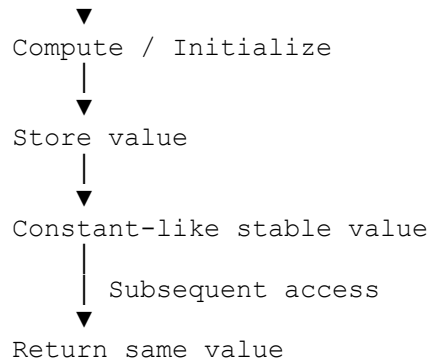
Status: *Preview feature.* “Lazy Constants” is designed to provide a way to declare values that are initialized **lazily**—only when first needed—while remaining stable thereafter. The feature builds on the `StableValue` API introduced through earlier previews.

1. Definition

Lazy Constants allow a value to be initialized on demand rather than when the containing class or object is initialized.

The basic lifecycle is:

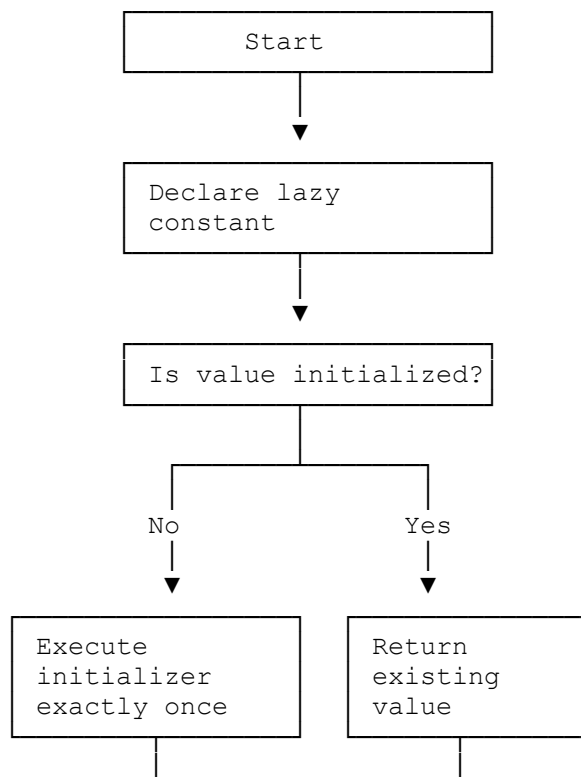


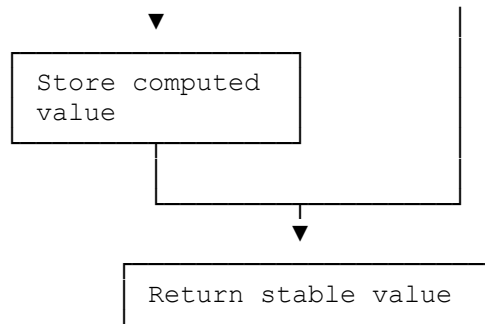


This is useful when initialization is **expensive, rarely needed**, or depends on information that is not available until later.

The important distinction is that a lazy constant is not simply a normal mutable variable. Once initialized, its value is intended to remain **stable**.

2. Flowchart





3. Java Code Example

A conceptual example using the `StableValue` API looks like this:

```
import java.util.concurrent.StableValue;

public class LazyConstants {

    private static final StableValue<String> DATABASE_URL =
        StableValue.of();

    private static String initializeDatabaseUrl() {

        System.out.println(
            "Initializing database URL..."
        );

        // Expensive initialization
        return "jdbc:mysql://localhost:3306/mydb";
    }

    public static String getDatabaseUrl() {

        return DATABASE_URL.orElseSet(
            LazyConstants::initializeDatabaseUrl
        );
    }

    public static void main(String[] args) {

        System.out.println("Application started.");

        // First access:
        // initializer is executed.
        System.out.println(
            getDatabaseUrl()
        );

        // Second access:
        // previously initialized value is returned.
    }
}
```

```

        System.out.println(
            getDatabaseUrl()
        );
    }
}

```

Conceptual output

```

Application started.
Initializing database URL...
jdbc:mysql://localhost:3306/mydb
jdbc:mysql://localhost:3306/mydb

```

Notice that:

```

getDatabaseUrl() → First call
                  ↓
            Initialize value
                  ↓
            Store value
                  ↓
            Return value

getDatabaseUrl() → Second call
                  ↓
            Value already exists
                  ↓
            Return same value

```

4. Lazy Initialization vs Normal Initialization

Normal initialization

```

public class NormalConstant {

    private static final String DATABASE_URL =
        initializeDatabaseUrl();

    private static String initializeDatabaseUrl() {

        System.out.println("Initializing...");

        return "jdbc:mysql://localhost:3306/mydb";
    }
}

```

The initialization happens as part of class initialization.

Lazy initialization

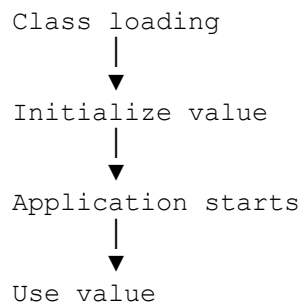
```
private static final StableValue<String> DATABASE_URL =
    StableValue.of();

static String getDatabaseUrl() {
    return DATABASE_URL.orElseSet(
        LazyConstants::initializeDatabaseUrl
    );
}
```

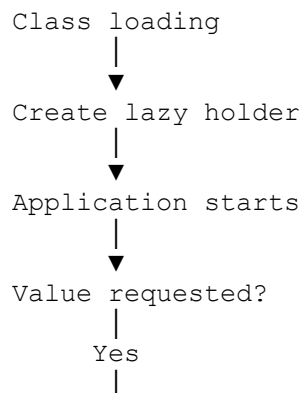
Here, initialization is deferred until `getDatabaseUrl()` actually needs the value.

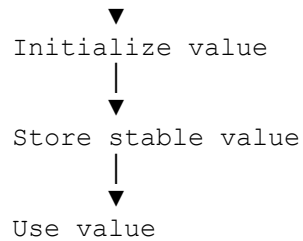
5. Execution Comparison

NORMAL CONSTANT



LAZY CONSTANT





6. Why Lazy Constants Are Useful

Situation	Benefit
Expensive initialization	Work is postponed until needed
Rarely used configuration	Avoid unnecessary initialization
Large objects	Object isn't created prematurely
Environment-dependent values	Initialization can happen when the value is actually needed
Threaded applications	Stable-value semantics provide safe publication
Repeated access	Once initialized, the same value can be reused
Cleaner lazy initialization	Avoids writing custom synchronization logic

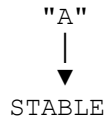
7. Important Property

A normal mutable variable can change:

```
String value = "A";  
  
value = "B";  
value = "C";
```

A lazy constant conceptually follows:

```
UNINITIALIZED  
↓  
▼
```



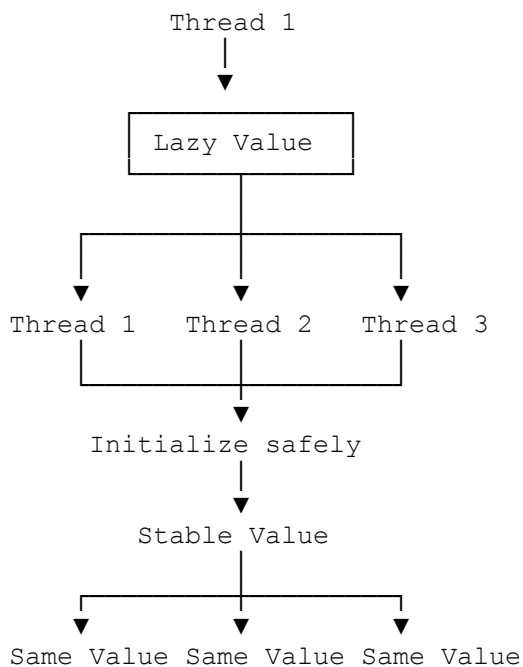
It does **not** follow:

"A" → "B" → "C"

This gives the JVM and compiler stronger information about the value's stability.

8. Thread-Safety Concept

Consider multiple threads requesting the value simultaneously:



The API is intended to provide safe, efficient lazy initialization without requiring developers to implement their own locking pattern.

9. Key Difference

`final`

- Prevents reassignment
- Does not inherently mean lazy initialization

`StableValue / Lazy Constant`

- Supports lazy initialization
- Value becomes stable after initialization
- Designed to provide safe publication

In one sentence

Lazy Constants provide a Java-native mechanism for creating a value **only when it is first requested, then treating that value as stable for subsequent accesses**, reducing unnecessary initialization while retaining strong immutability/stability semantics.

Java 27 — Structured Concurrency (Seventh Preview)

Java Version: 27

Year: 2026

Feature: Structured Concurrency (Seventh Preview)

UniqueClassName: `StructuredConcurrency`

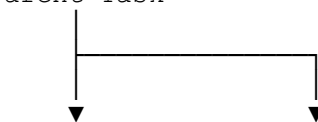
Status: Preview feature. Structured Concurrency treats related concurrent tasks as a single unit of work, so their lifetimes, cancellation, failure handling, and results are managed within a well-defined lexical scope.

1. Definition

Structured Concurrency provides an API for running multiple concurrent subtasks while maintaining a clear parent-child relationship between them.

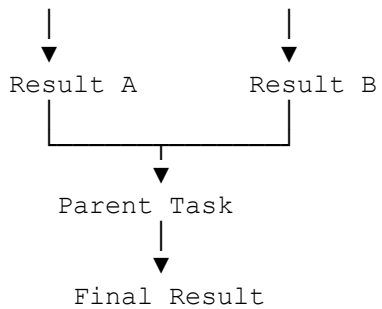
Instead of creating independent threads that can continue running after the initiating operation has finished, structured concurrency establishes a **structured scope**:

Parent Task



Child Task A

Child Task B

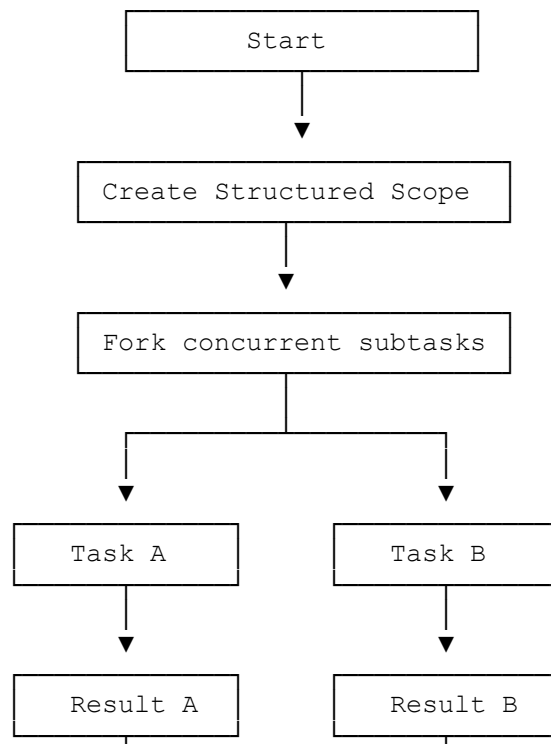


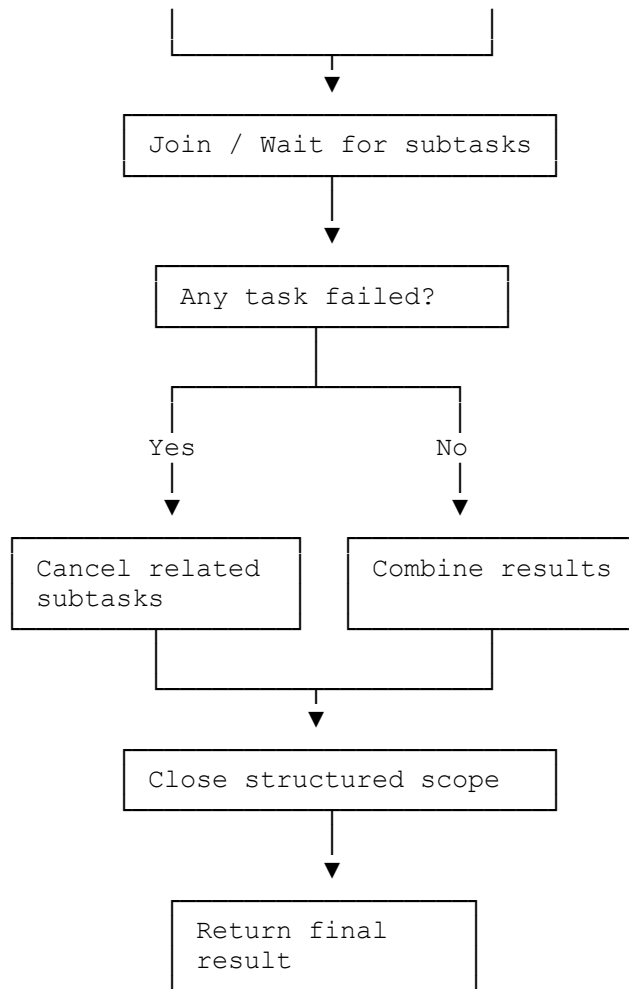
The parent does not normally finish the operation until the relevant child tasks have completed or the scope has been cancelled.

This makes concurrent code easier to reason about because:

- Child tasks are bounded by their parent scope.
 - Failures can be propagated systematically.
 - Cancellation can be propagated to related tasks.
 - Results can be gathered in one place.
 - Threads do not become accidentally detached from the operation that created them.
-

2. Flowchart





3. Java Code Example

A conceptual Java 27 example can use `StructuredTaskScope`:

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    static String getUser() throws InterruptedException {
        Thread.sleep(1000);
        return "User: Anurag";
    }

    static String getOrders() throws InterruptedException {
        Thread.sleep(1500);
        return "Orders: 5";
    }
}
```

```

public static void main(String[] args)
    throws InterruptedException {

    try (var scope =
        StructuredTaskScope.open()) {

        var userTask =
            scope.fork(StructuredConcurrency::getUser);

        var ordersTask =
            scope.fork(StructuredConcurrency::getOrders);

        scope.join();

        String user = userTask.get();
        String orders = ordersTask.get();

        System.out.println(user);
        System.out.println(orders);
    }
}

```

Conceptual output

User: Anurag
Orders: 5

The important part is:

```

var userTask =
    scope.fork(StructuredConcurrency::getUser);

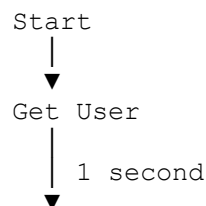
var ordersTask =
    scope.fork(StructuredConcurrency::getOrders);

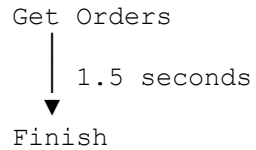
```

Both operations can execute concurrently within the same structured scope.

4. Sequential vs Structured Concurrent Execution

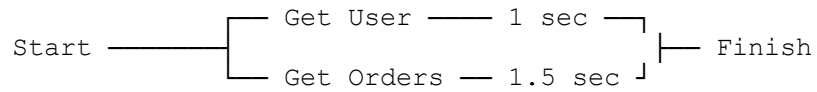
Traditional sequential approach





Approximate total: 2.5 seconds

Structured concurrency

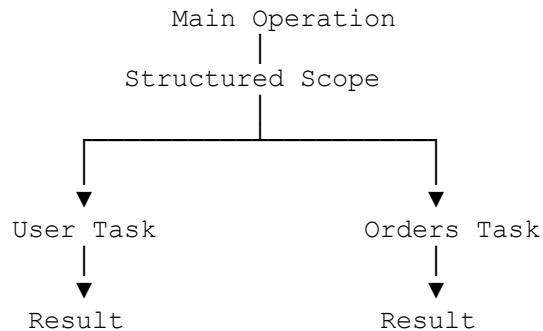


Approximate total: 1.5 seconds

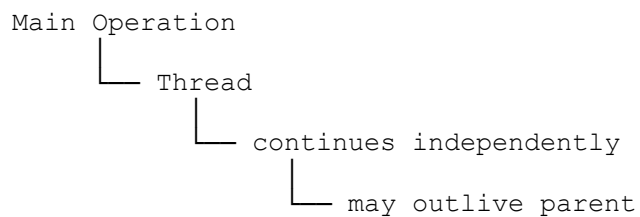
The exact runtime depends on scheduling and execution conditions, but the independent operations can overlap.

5. Parent-Child Relationship

Structured concurrency creates an important hierarchy:



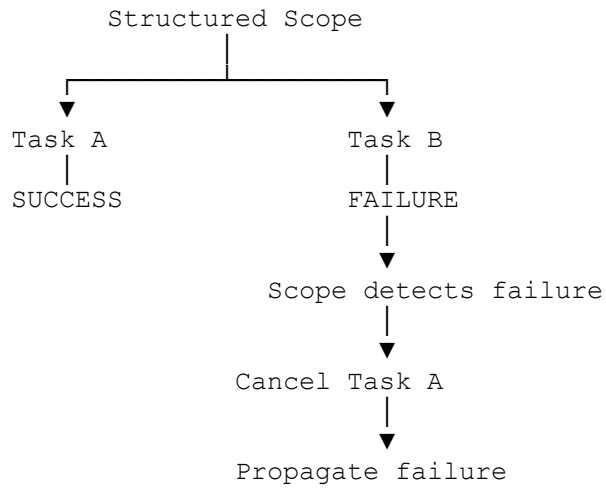
This is different from creating unrelated background tasks:



Structured concurrency attempts to avoid that kind of accidental lifetime mismatch.

6. Failure and Cancellation

Suppose two tasks are running:



For example:

```
try (var scope =  
    StructuredTaskScope.open()) {  
  
    var task1 =  
        scope.fork(() -> fetchCustomer());  
  
    var task2 =  
        scope.fork(() -> fetchAccount());  
  
    scope.join();  
  
    // Collect results here  
}
```

The scope provides a natural place to coordinate:

```
fork()  
↓  
execute  
↓  
join()  
↓  
handle result/failure  
↓  
close()
```

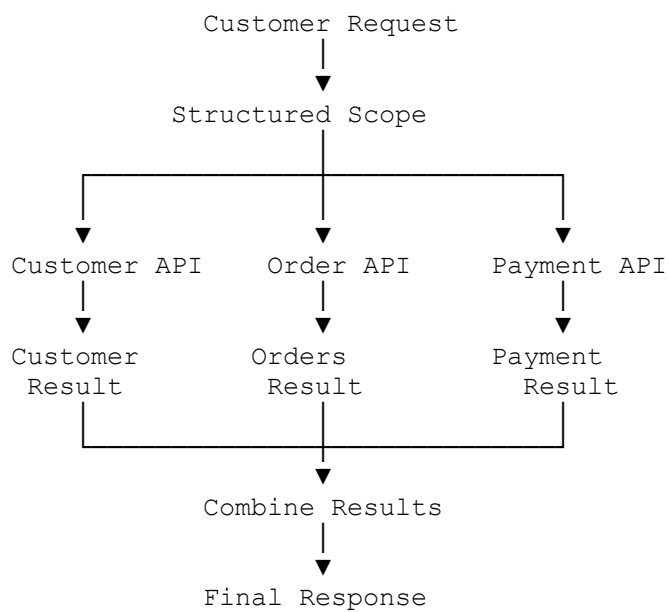
7. Example: Parallel Web-Service Operations

Imagine an application that needs:

Customer information
+
Order information
+
Payment information

These operations are independent.

Structured concurrency can model them as:



Example:

```
import java.util.concurrent.StructuredTaskScope;

public class CustomerDashboard {

    static String fetchCustomer() {
        return "Customer information";
    }

    static String fetchOrders() {
        return "Order information";
    }

    static String fetchPayments() {
        return "Payment information";
    }

    public static void main(String[] args)
```

```

        throws InterruptedException {

    try (var scope =
        StructuredTaskScope.open()) {

        var customer =
            scope.fork(CustomerDashboard::fetchCustomer);

        var orders =
            scope.fork(CustomerDashboard::fetchOrders);

        var payments =
            scope.fork(CustomerDashboard::fetchPayments);

        scope.join();

        System.out.println(customer.get());
        System.out.println(orders.get());
        System.out.println(payments.get());
    }
}

```

8. Important APIs

The Structured Concurrency API revolves around a structured task scope and operations such as:

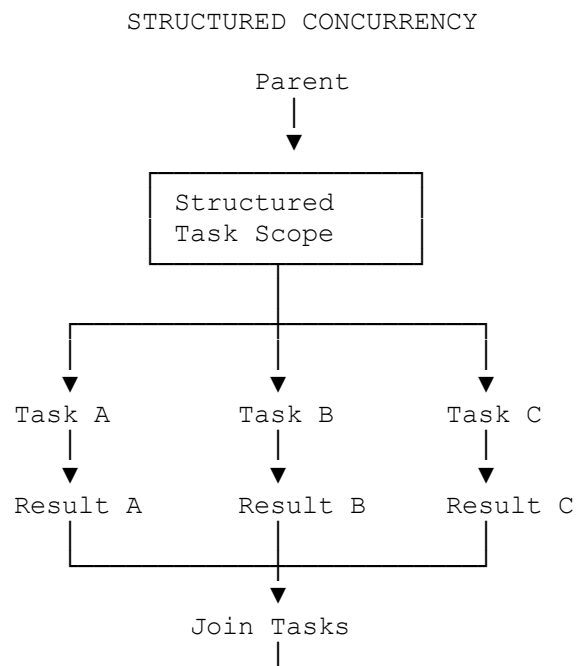
Operation	Purpose
StructuredTaskScope.open()	Creates a structured concurrency scope
fork(...)	Starts a concurrent subtask
join()	Waits for subtasks
get()	Obtains a successful subtask result
close()	Closes the scope
Shutdown/cancellation policies	Coordinate failures and cancellation

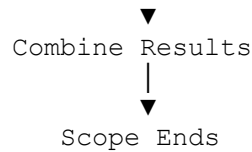
The exact API available in Java 27 should be checked against the final JDK 27 documentation because this feature is a **preview API**, and preview APIs can change between releases.

9. Structured Concurrency vs Traditional Threads

Aspect	Traditional Threads	Structured Concurrency
Task relationship	Often implicit	Explicit parent-child relationship
Lifetime	Can become detached	Bound to scope
Cancellation	Developer-managed	Scope-oriented
Failure handling	Often scattered	Centralized
Result collection	Developer-managed	Scope-based
Code structure	Can become difficult to follow	Mirrors concurrent task structure
Resource management	Manual	Scope-oriented
Readability	Potentially complex	More structured

10. Core Concept





Key takeaway

Structured Concurrency makes concurrent programming follow the same principle as structured programming: **tasks are created inside a well-defined scope, their completion and failures are coordinated within that scope, and the parent operation does not accidentally lose track of its child tasks.**

Java 27 — Compact Object Headers by Default

Java Version: 27

Year: 2026

Feature: Compact Object Headers by Default

UniqueClassName: CompactObjectHeaders

Status note: Compact Object Headers were introduced as a preview feature in JDK 24 and subsequently became a permanent/default capability in later JDK development. The important idea is that the JVM uses a smaller object header representation to reduce per-object memory overhead. The Java application code generally does **not** directly manipulate the object header; this is a JVM implementation optimization.

1. Definition

Every Java object has JVM-level metadata associated with it. Traditionally, an object header can contain information such as:

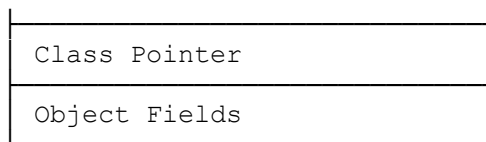
- Object identity/hash-code information
- Garbage-collection state
- Class/type information
- Locking/synchronization state

Compact Object Headers reduce the amount of memory required for this metadata.

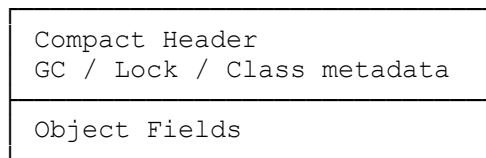
Conceptually:

Traditional Object

Object Header
Mark / GC / Lock information

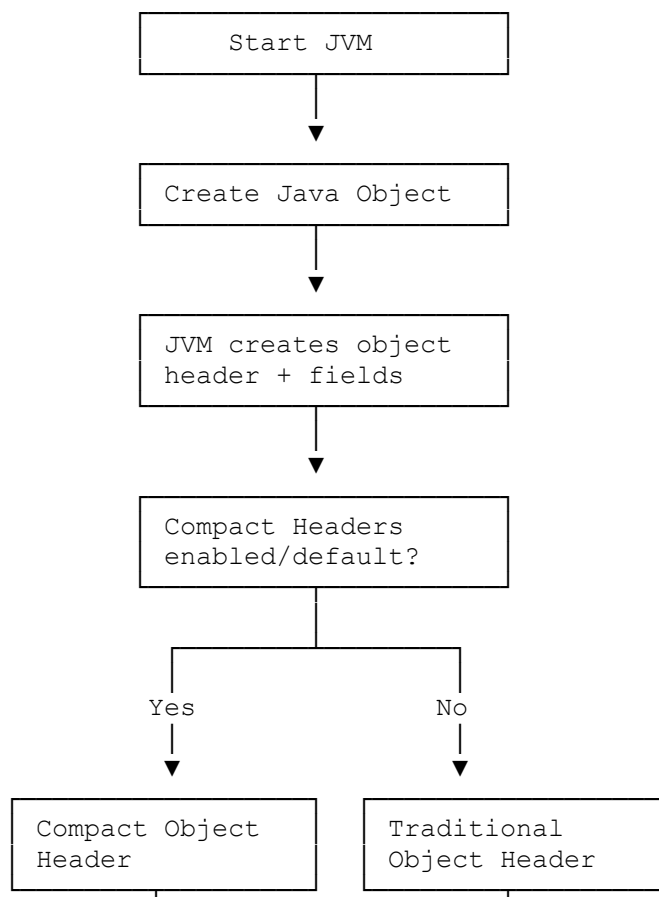


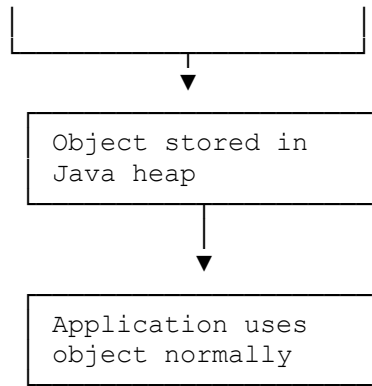
Compact Object Header



The main benefit is **lower memory consumption per object**, which can become significant when an application creates millions or billions of small objects.

2. Flowchart





3. Java Code Example

Because compact object headers are a **JVM implementation feature**, ordinary Java code does not need to be rewritten.

For example:

```
public class CompactObjectHeaders {  
  
    private int id;  
    private String name;  
  
    public CompactObjectHeaders(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    public static void main(String[] args) {  
  
        CompactObjectHeaders object =  
            new CompactObjectHeaders(  
                101,  
                "Anurag"  
            );  
  
        System.out.println(  
            "ID    : " + object.id  
        );  
  
        System.out.println(  
            "Name : " + object.name  
        );  
    }  
}
```

Output

ID : 101
Name : Anurag

The source code is essentially identical whether the JVM uses compact or traditional headers.

The optimization occurs **inside the JVM object representation**.

4. Creating Many Objects

Compact headers become particularly interesting when applications create huge numbers of small objects.

```
public class CompactObjectHeaders {  
    static class Customer {  
        int id;  
  
        Customer(int id) {  
            this.id = id;  
        }  
    }  
  
    public static void main(String[] args) {  
        int numberOfObjects = 10_000_000;  
  
        Customer[] customers =  
            new Customer[numberOfObjects];  
  
        for (int i = 0; i < numberOfObjects; i++) {  
            customers[i] = new Customer(i);  
        }  
  
        System.out.println(  
            "Created " +  
            numberOfObjects +  
            " Customer objects."  
        );  
    }  
}
```

Output:

Created 10000000 Customer objects.

Each Customer object has JVM metadata in addition to its application fields. Reducing that per-object metadata can produce meaningful heap savings at this scale.

5. Traditional vs Compact Object Representation

Conceptually:

TRADITIONAL

```
Object 1 → Header + Fields
Object 2 → Header + Fields
Object 3 → Header + Fields
Object 4 → Header + Fields
Object 5 → Header + Fields
...
Object N → Header + Fields
```

COMPACT

```
Object 1 → Compact Header + Fields
Object 2 → Compact Header + Fields
Object 3 → Compact Header + Fields
Object 4 → Compact Header + Fields
Object 5 → Compact Header + Fields
...
Object N → Compact Header + Fields
```

The savings are **per object**, so the aggregate effect grows with the number of objects.

6. Why Object Headers Matter

Consider an application containing:

```
1,000 objects
    ↓
small overhead

1,000,000 objects
    ↓
noticeable overhead

100,000,000 objects
    ↓
potentially substantial overhead
```

This is especially relevant to applications involving:

- Large collections
 - Caches
 - Object-heavy enterprise applications
 - Graph structures
 - Framework-generated objects
 - Microservices
 - High-throughput servers
 - In-memory databases
 - Large-scale data processing
-

7. Conceptual Memory Layout

Suppose a simplified object contains:

```
class Employee {  
    int id;  
    int age;  
}
```

Conceptually:

Traditional:

Object Header
Class / JVM metadata
int id
int age
Alignment / padding

Compact:

Compact Object Header
int id
int age
Alignment / padding

The exact physical layout depends on the JVM, architecture, object alignment, compressed references, and other implementation details.

8. Important Point: Java Code Does Not See the Header

The following:

```
Employee employee =  
    new Employee(101, 30);
```

doesn't expose:

```
Object Header  
Mark Word  
Class Metadata  
GC Bits  
Lock State
```

to normal Java code.

Those are JVM-level implementation details.

Therefore, developers generally benefit from compact headers **without changing application source code**.

9. Memory-Saving Concept

The simplified relationship is:

```
Total Object Memory  
    =  
Object Header  
+  
Object Fields  
+  
Alignment / Padding
```

With compact headers:

```
Total Object Memory  
    =  
Smaller Object Header  
+  
Object Fields
```

+
Alignment / Padding

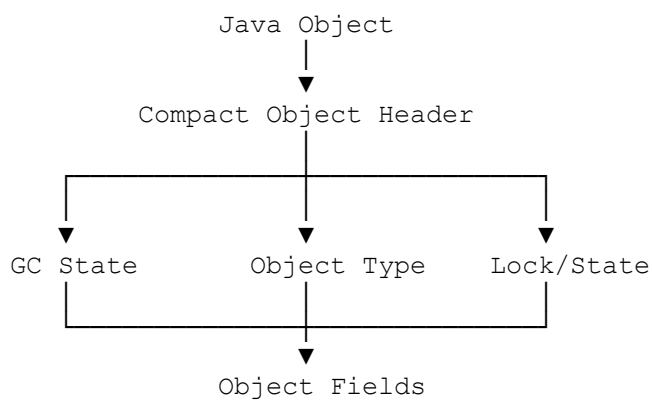
Therefore:

More objects
x
Smaller per-object overhead
↓
Potentially lower heap usage

10. Compact Object Headers and Garbage Collection

Object headers contain information important to JVM operations, including garbage collection and object state.

Conceptually:



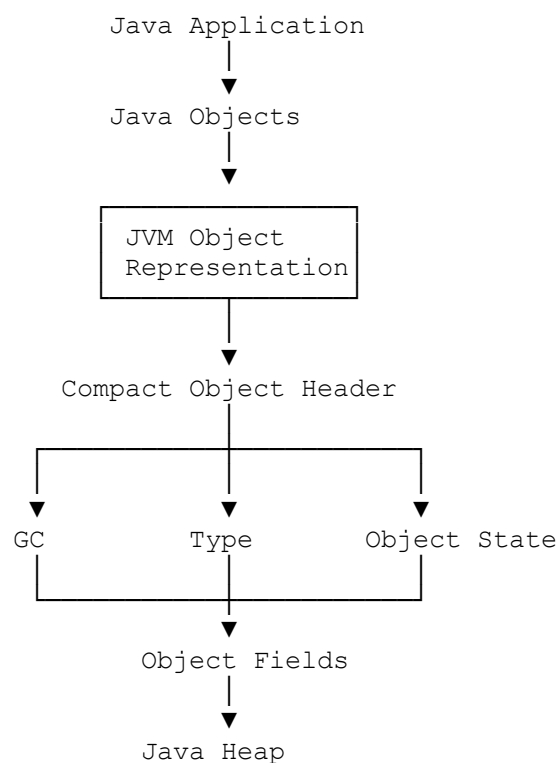
The JVM manages these details transparently.

11. Advantages

Advantage	Description
Lower memory overhead	Smaller metadata associated with objects
Better heap utilization	More application objects can fit into the same heap
Large object populations	Especially useful when there are millions of objects

Potential cache benefits	Smaller objects can improve memory locality
Less memory pressure	More heap space remains available for application data
No source-code changes	JVM-level optimization
Potential GC benefits	Lower object footprint can reduce overall heap pressure

12. Conceptual Architecture



Key takeaway

Compact Object Headers reduce the memory overhead associated with each Java object by using a more compact JVM-level header representation. With **Compact Object Headers by default**, applications can potentially store significantly more small objects in the same heap without requiring changes to ordinary Java source code.

The feature is therefore primarily a **JVM memory-layout optimization**, rather than a new Java language construct.

Java 27 — JFR In-Process Data Redaction

Java Version: 27

Year: 2026

Feature: JFR In-Process Data Redaction

UniqueClassName: JFRInProcessDataRedaction

Status note: This is a **JDK/JFR security and observability capability**, rather than a normal Java language feature. The goal is to allow sensitive information to be redacted **before it becomes part of Java Flight Recorder (JFR) recording data**, reducing the chance that secrets or personally identifiable information (PII) are captured in diagnostic recordings.

1. Definition

JFR In-Process Data Redaction means sensitive data can be filtered or replaced **inside the running JVM/application before JFR records or exposes the information**.

For example, an application might process:

```
User:
Anurag Bharti

Email:
anurag@example.com

Token:
eyJhbGciOi...
```

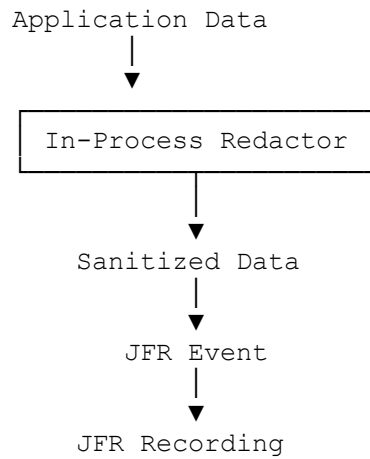
Instead of allowing sensitive values into a diagnostic event:

```
User: Anurag Bharti
Email: anurag@example.com
Token: eyJhbGciOi...
```

the information could be transformed into:

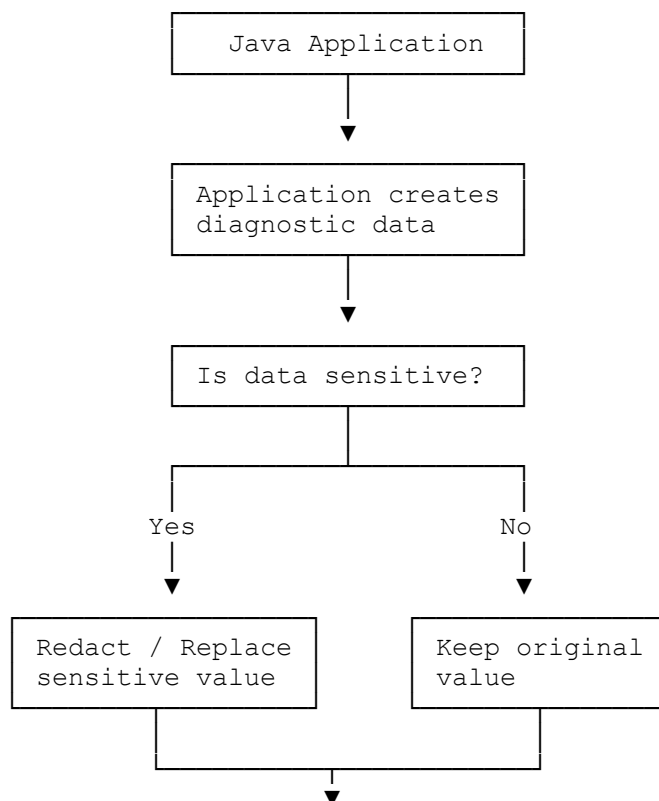
```
User: [REDACTED]
Email: [REDACTED]
Token: [REDACTED]
```

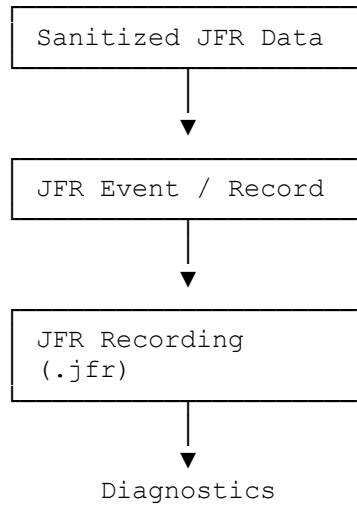
The conceptual security boundary is:



The important principle is **redact before recording**, rather than relying on someone to sanitize a .jfr recording afterward.

2. Flowchart





3. Java Code Example

The following example demonstrates the **concept** using a custom JFR event and redaction before assigning sensitive values to the event.

```
import jdk.jfr.Event;
import jdk.jfr.Label;
import jdk.jfr.Name;

public class JFRInProgressDataRedaction {

    @Name("application.UserActivity")
    @Label("User Activity")
    static class UserActivityEvent extends Event {

        @Label("User")
        String user;

        @Label("Email")
        String email;

        @Label("Access Token")
        String accessToken;
    }

    public static void main(String[] args) {

        String user = "Anurag Bharti";
        String email = "anurag@example.com";
        String token = "secret-access-token-123";

        UserActivityEvent event =
            new UserActivityEvent();
```

```

        event.user = redactUser(user);
        event.email = redactEmail(email);
        event.accessToken = redactSecret(token);

        event.commit();

        System.out.println("JFR event committed.");
    }

    private static String redactUser(String value) {
        return "[REDACTED]";
    }

    private static String redactEmail(String value) {
        return "[REDACTED]";
    }

    private static String redactSecret(String value) {
        return "[REDACTED]";
    }
}

```

The key principle is:

```
event.accessToken = redactSecret(token);
```

rather than:

```
event.accessToken = token;
```

So the sensitive value never needs to be placed into that JFR event.

4. Example Without Redaction

This is potentially dangerous:

```

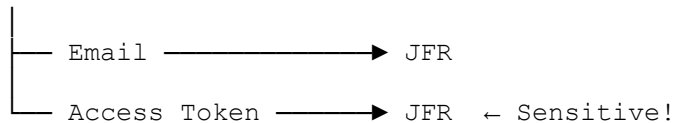
event.user = user;
event.email = email;
event.accessToken = token;

event.commit();

```

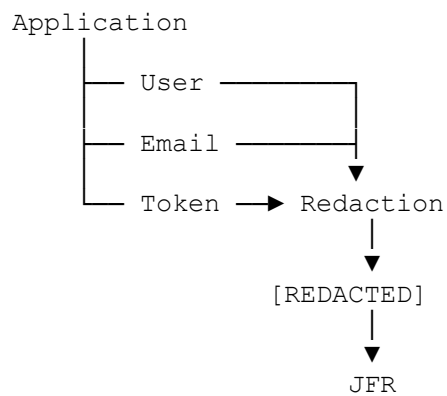
Conceptually:





The recording can then contain information that was never intended to be exposed through diagnostics.

5. Example With In-Process Redaction



This provides a stronger security boundary because the sensitive value is transformed before being committed to the recording.

6. Practical Redaction Function

A more realistic example can preserve limited information while hiding the sensitive portion:

```
private static String redactEmail(String email) {  
    int at = email.indexOf('@');  
  
    if (at <= 1) {  
        return "[REDACTED]";  
    }  
  
    return email.charAt(0)  
        + "***"  
        + email.substring(at);  
}
```

For example:

Original:
anurag@example.com

Redacted:
a***@example.com

For credentials, however, complete removal is generally safer:

```
private static String redactSecret(String secret) {  
    return "[REDACTED]";  
}
```

7. Sensitive Data Categories

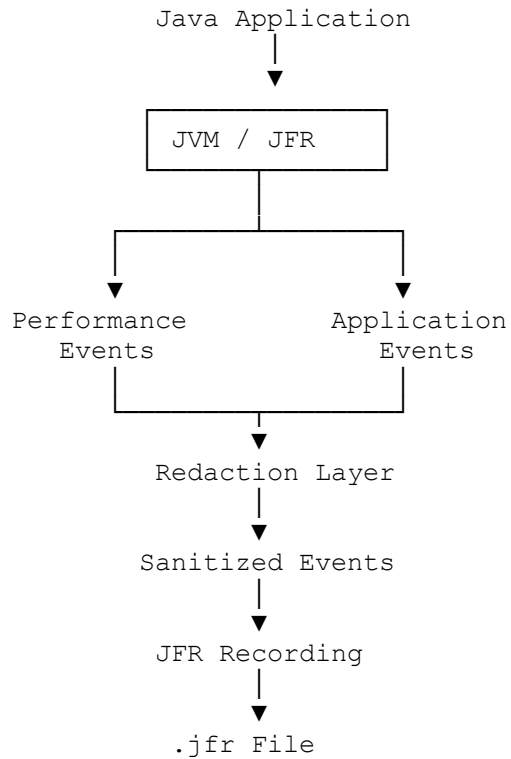
A redaction policy might identify:

Data	Example	JFR treatment
Password	MyPassword123	[REDACTED]
Access token	eyJhbGci...	[REDACTED]
API key	sk-...	[REDACTED]
Session ID	SID-123456	[REDACTED]
Credit-card number	4111...	[REDACTED]
Email	user@example.com	Mask/redact
Phone number	+91...	Mask/redact
User ID	USER-123	Depends on policy
Stack trace	com.example...	Usually retain
GC information	JVM data	Usually retain

8. JFR Recording Architecture

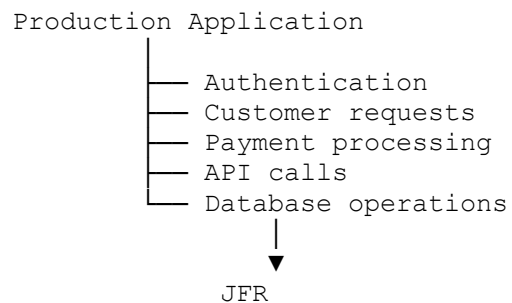
JFR sits inside the JVM and collects events for diagnostics and performance analysis.

Conceptually:



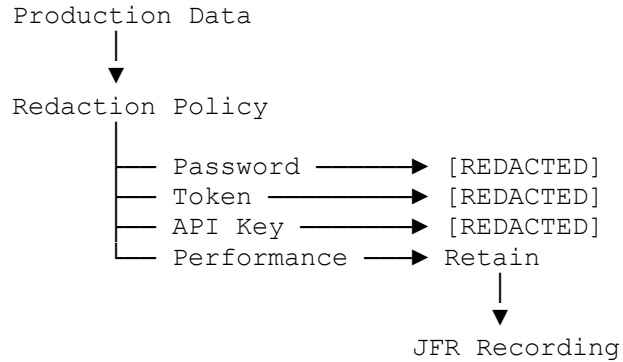
9. Why In-Process Redaction Matters

Consider a production server:



Diagnostic recordings are extremely useful, but they can become sensitive artifacts themselves.

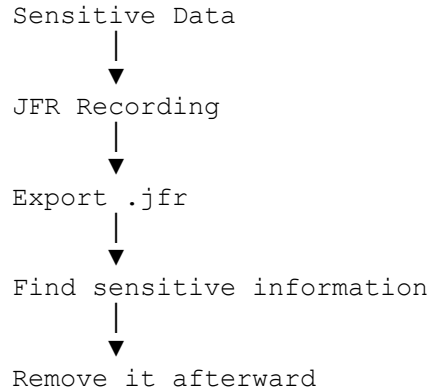
With in-process redaction:



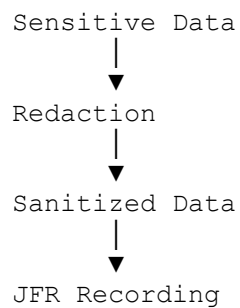
This allows developers and operations teams to obtain useful diagnostic information while reducing unnecessary exposure of sensitive application data.

10. Key Difference

Traditional approach

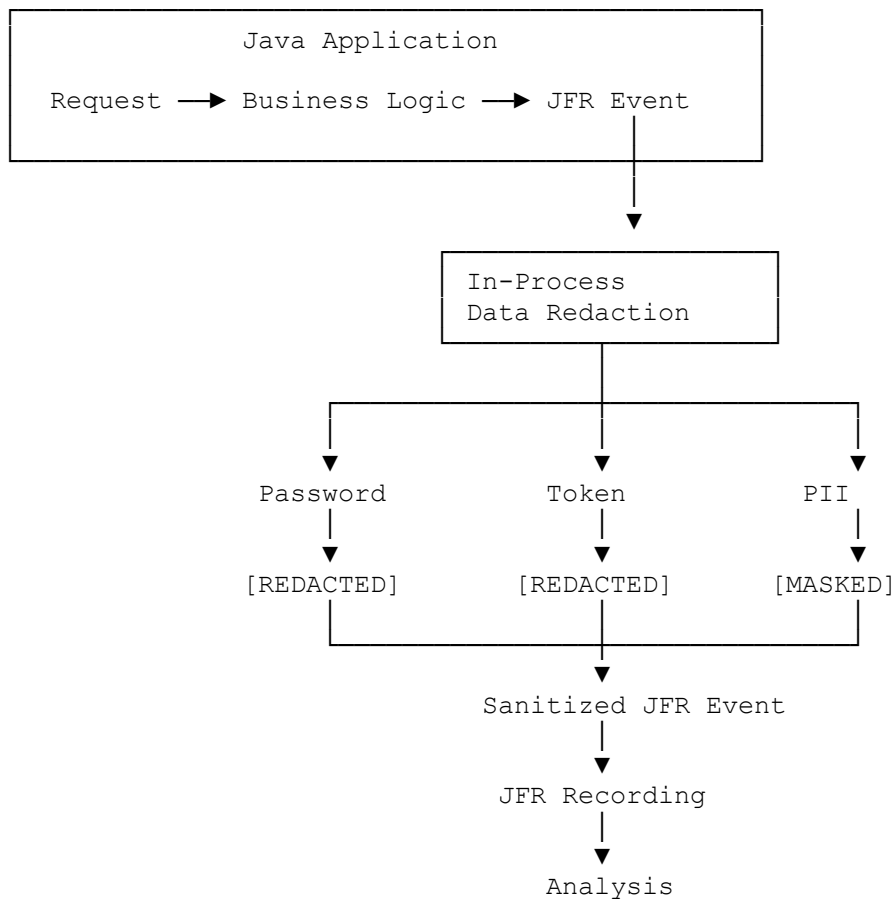


In-process redaction



The second approach is preferable when the goal is to prevent sensitive information from entering the diagnostic artifact in the first place.

11. Core Architecture



Key takeaway

JFR In-Process Data Redaction is about putting the privacy/security boundary **inside the running JVM**, so sensitive information can be removed or transformed before it becomes part of JFR diagnostic data. This is particularly valuable for production systems where JFR recordings may be collected, transferred, stored, or shared for troubleshooting.

Java 27 — Vector API (Twelfth Incubator)

Java Version: 27

Year: 2026

Feature: Vector API (Twelfth Incubator)

UniqueClassName: VectorAPI

Status: Incubator feature. The Vector API provides Java APIs for expressing vector computations that the JIT compiler can map to SIMD (Single Instruction, Multiple Data) hardware instructions when available.

1. Definition

The **Vector API** allows Java developers to perform operations on multiple values simultaneously instead of processing each value individually.

Traditional scalar computation

```
A1 + B1 → C1  
A2 + B2 → C2  
A3 + B3 → C3  
A4 + B4 → C4
```

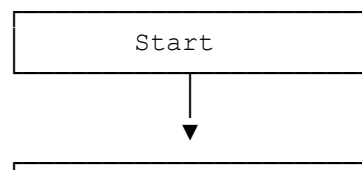
Each operation is performed separately.

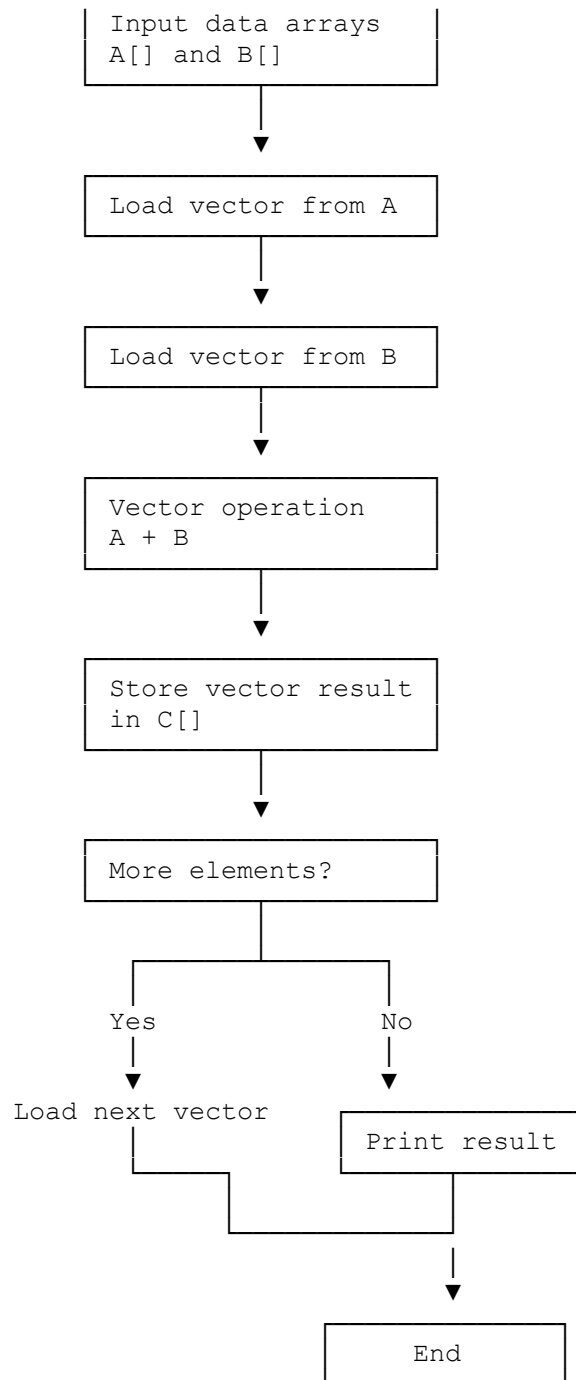
Vector computation

```
A: [A1 A2 A3 A4]  
B: [B1 B2 B3 B4]  
  ↓  
  Vector Addition  
  ↓  
C: [C1 C2 C3 C4]
```

This can allow the CPU to process several values using a single SIMD instruction.

2. Flowchart





3. Java Code Example

The following example demonstrates vector addition using the Vector API.

```

import jdk.incubator.vector.IntVector;
import jdk.incubator.vector.VectorSpecies;

public class VectorAPI {

    private static final VectorSpecies<Integer> SPECIES =
        IntVector.SPECIES_PREFERRED;

    public static void main(String[] args) {

        int[] a = {
            1, 2, 3, 4,
            5, 6, 7, 8
        };

        int[] b = {
            10, 20, 30, 40,
            50, 60, 70, 80
        };

        int[] result = new int[a.length];

        for (int i = 0;
            i < a.length;
            i += SPECIES.length()) {

            var mask =
                SPECIES.indexInRange(i, a.length);

            var va =
                IntVector.fromArray(
                    SPECIES,
                    a,
                    i,
                    mask
                );

            var vb =
                IntVector.fromArray(
                    SPECIES,
                    b,
                    i,
                    mask
                );

            var vc =
                va.add(vb);

            vc.intoArray(
                result,
                i,
                mask
            );
        }

        System.out.println("Vector Result:");
    }
}

```

```

        for (int value : result) {
            System.out.print(value + " ");
        }
    }
}

```

Output

Vector Result:
11 22 33 44 55 66 77 88

4. How the Vector Operation Works

The important statement is:

```
var vc = va.add(vb);
```

Conceptually:

```

va = [1  2  3  4]
vb = [10 20 30 40]
    |
    ▼
  add()
    |
    ▼
vc = [11 22 33 44]

```

The operation applies the same addition to all vector lanes.

5. Scalar vs Vector API

Scalar Java

```

for (int i = 0; i < a.length; i++) {
    result[i] = a[i] + b[i];
}

```

Conceptually:

```

1  + 10 = 11
2  + 20 = 22
3  + 30 = 33

```

4 + 40 = 44

Vector API

```
var va = IntVector.fromArray(...);  
var vb = IntVector.fromArray(...);  
  
var vc = va.add(vb);
```

Conceptually:

```
[1 2 3 4] + [10 20 30 40]  
           |  
           ▼  
[11 22 33 44]
```

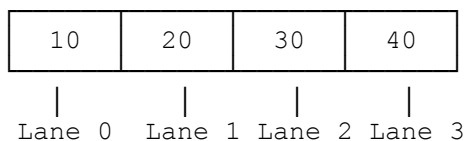
The JIT compiler can map suitable Vector API operations to hardware SIMD instructions.

6. Vector Lanes

A vector consists of multiple **lanes**.

For example:

Vector<Integer>



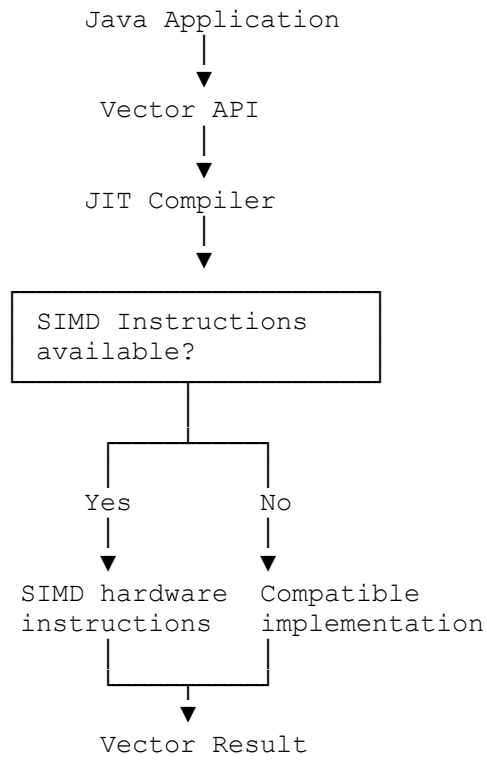
The number of lanes depends on the selected vector species and the underlying hardware.

This is why the example uses:

IntVector.SPECIES_PREFERRED

rather than assuming a fixed vector width.

7. Vector API Architecture



The API therefore provides a portable Java programming model while allowing the runtime to exploit the capabilities of the target CPU.

8. Example: Vector Multiplication

The API is not limited to addition.

```
var va =
    IntVector.fromArray(
        SPECIES,
        a,
        i,
        mask
    );

var vb =
    IntVector.fromArray(
        SPECIES,
        b,
        i,
        mask
    );
```

```
var result =  
    va.mul(vb);
```

Conceptually:

```
[2  4  6  8]  
  x  
[3  5  7  9]  
  |  
  ▼  
[6 20 42 72]
```

Other vector operations include operations such as:

```
add()  
sub()  
mul()  
div()  
min()  
max()  
and()  
or()  
xor()
```

and comparisons and rearrangements.

9. Masking

The Vector API can use a **VectorMask** when the array length is not an exact multiple of the vector length.

For example:

```
Array length = 10  
Vector length = 4
```

```
Iteration 1:  
[0 1 2 3]
```

```
Iteration 2:  
[4 5 6 7]
```

```
Iteration 3:  
[8 9 X X]  
    ↑  
  masked
```

That's why the example uses:

```
var mask = SPECIES.indexInRange(i, a.length);
```

This prevents operations from accessing elements outside the valid array range.

10. Typical Use Cases

Use case	Example
Numerical computing	Matrix/vector calculations
Image processing	Pixel operations
Signal processing	Audio/video processing
Machine learning	Numerical transformations
Cryptography	Parallel arithmetic
Scientific computing	Simulations
Data processing	Array transformations
Statistics	Parallel aggregation
Financial calculations	Large numerical datasets

11. Performance Concept

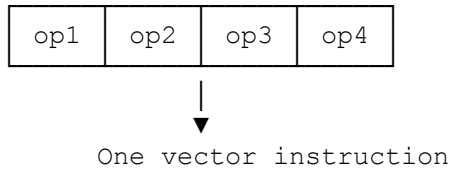
Suppose a CPU vector instruction can process four integers at once.

Scalar

```
1 operation
↓
1 operation
↓
1 operation
↓
```

1 operation

SIMD

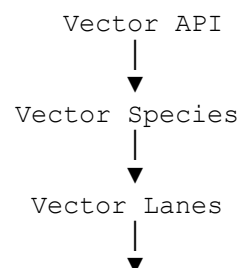


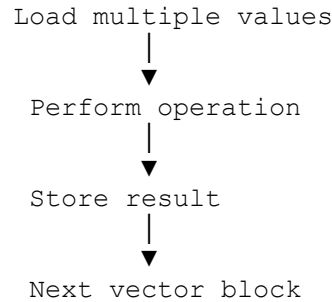
This **does not guarantee a 4× speedup**. Actual performance depends on CPU architecture, memory bandwidth, JIT compilation, data size, algorithm, and other factors.

12. Vector API vs Ordinary Streams

Feature	Stream API	Vector API
Primary goal	Express data processing	Express SIMD computation
Parallelism	Logical/data processing	CPU vector lanes
SIMD control	Indirect	Explicit vector operations
Typical data	Objects/collections	Primitive numerical data
Main benefit	Declarative processing	Hardware-efficient numerical computation

13. Core Concept





Key takeaway

The Vector API enables Java applications to express computations over multiple primitive values simultaneously. As an incubator API, it is intended to provide a portable way for Java code to take advantage of modern SIMD hardware while allowing the JIT compiler to select appropriate machine instructions for the target platform.

Java 27 — Primitive Types in Patterns, instanceof, and switch (Fifth Preview)

Java Version: 27

Year: 2026

Feature: Primitive Types in Patterns, instanceof, and switch (Fifth Preview)

UniqueClassName: PrimitiveTypesPatterns

Status: Preview feature. It extends Java's pattern-matching model so that **primitive values can participate directly in type patterns, instanceof, and switch**, including useful conversions between primitive types where permitted.

1. Definition

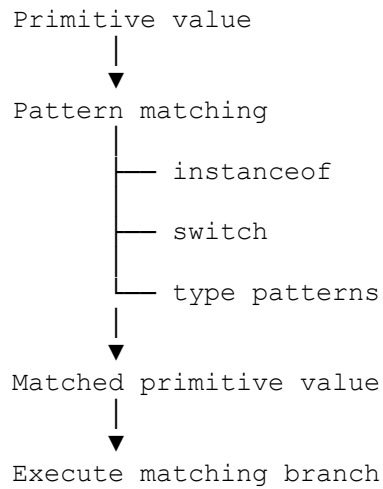
Traditionally, Java pattern matching primarily worked with **reference types**:

```
if (value instanceof String s) {  
    System.out.println(s.length());  
}
```

Primitive values such as int, long, double, etc. were handled separately.

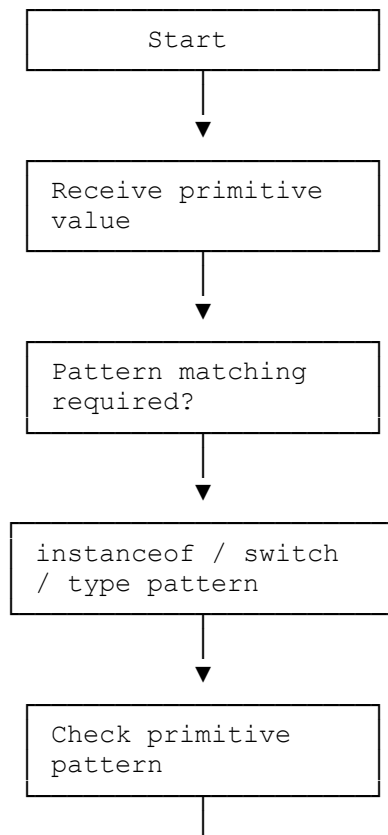
This feature expands pattern matching so that primitive types can be used directly in patterns.

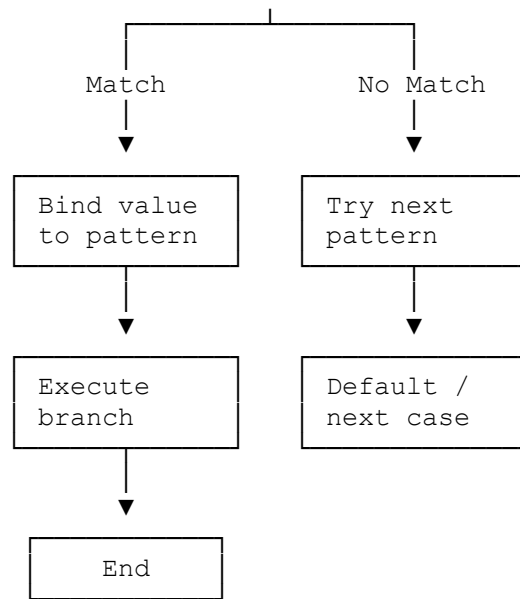
Conceptually:



The feature is particularly useful for making switch and pattern matching more expressive when dealing with numeric data.

2. Flowchart





3. Java Code Example — Primitive switch

A simple example is:

```
public class PrimitiveTypesPatterns {  
    public static void main(String[] args) {  
        int value = 100;  
        String result = switch (value) {  
            case 0 -> "Zero";  
            case int x when x > 0 && x < 100 ->  
                "Positive value below 100";  
            case int x when x >= 100 ->  
                "Positive value 100 or greater";  
            default ->  
                "Negative value";  
        };  
        System.out.println(result);  
    }  
}
```

Output

Positive value 100 or greater

Here:

```
case int x when x >= 100
```

uses a **primitive type pattern** to bind the int value to x.

4. Primitive instanceof

Primitive pattern matching can also be used with instanceof in contexts where the value can be tested against the applicable primitive pattern.

Conceptually:

```
int value = 42;

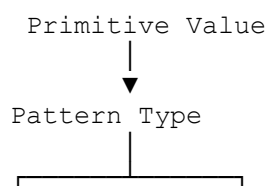
if (value instanceof int x) {
    System.out.println(
        "Primitive value: " + x
    );
}
```

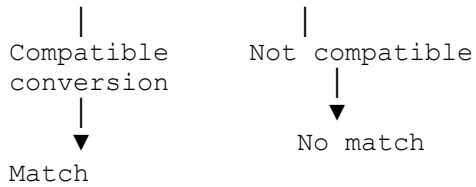
However, the precise legal forms and conversions are governed by the preview specification. The important concept is that primitive types can now participate in the pattern-matching model instead of requiring primitive-specific workarounds.

5. Pattern Matching with Numeric Types

The feature becomes especially interesting when conversions are involved.

Conceptually:





For example, a numeric value can be tested against an appropriate primitive type pattern when the conversion rules permit it.

6. switch with Multiple Primitive Patterns

Example:

```
public class PrimitiveTypesPatterns {  
    static String classify(int value) {  
        return switch (value) {  
            case 0 ->  
                "Zero";  
  
            case int x when x > 0 && x <= 10 ->  
                "Small positive number";  
  
            case int x when x > 10 && x <= 100 ->  
                "Medium positive number";  
  
            case int x when x > 100 ->  
                "Large positive number";  
  
            default ->  
                "Negative number";  
        };  
    }  
  
    public static void main(String[] args) {  
        System.out.println(classify(5));  
        System.out.println(classify(50));  
        System.out.println(classify(500));  
        System.out.println(classify(-10));  
    }  
}
```

Output

```
Small positive number  
Medium positive number  
Large positive number
```

Negative number

7. Traditional Approach vs Primitive Patterns

Traditional

```
int value = 50;

String result;

if (value > 0 && value <= 10) {
    result = "Small";
} else if (value > 10 && value <= 100) {
    result = "Medium";
} else {
    result = "Large";
}
```

Pattern-oriented approach

```
String result = switch (value) {

    case int x when x > 0 && x <= 10 ->
        "Small";

    case int x when x > 10 && x <= 100 ->
        "Medium";

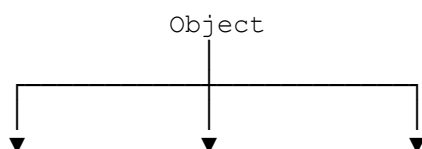
    default ->
        "Large";
};
```

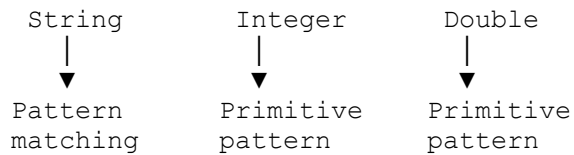
The latter integrates the test and value binding into the pattern.

8. Primitive Patterns with Reference Types

Pattern matching can also combine primitive and reference cases when the input is a reference type such as `Object`.

Conceptually:





For example, a future-style pattern-oriented switch can conceptually distinguish:

```
Object value = 42;
```

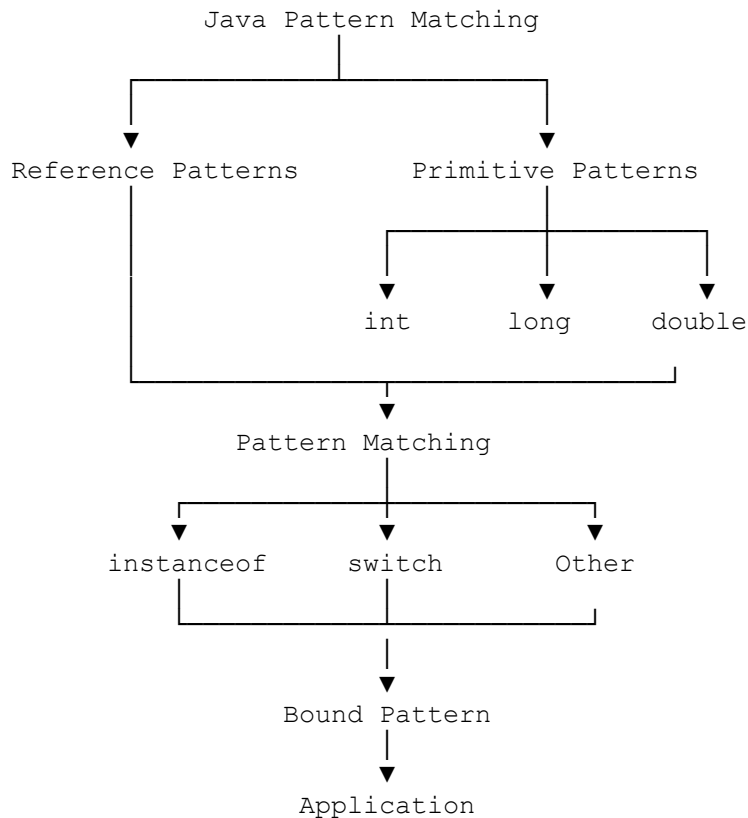
```
String result = switch (value) {  
    case String s ->  
        "String: " + s;  
  
    case Integer i ->  
        "Integer: " + i;  
  
    case Double d ->  
        "Double: " + d;  
  
    default ->  
        "Other";  
};
```

This is already possible through reference patterns and is related to the broader pattern-matching model. The Java 27 feature extends that model further to **primitive patterns themselves**.

9. Why Primitive Patterns Are Useful

Benefit	Explanation
Cleaner numeric code	Primitive values can participate directly in pattern matching
Better switch expressions	Numeric classification can be expressed naturally
Unified pattern model	Primitive and reference patterns become more consistent
Less conversion boilerplate	Fewer manual casts/conversions
Value binding	Pattern variables can directly hold matched values
Better readability	Conditions and matching logic can be expressed together

10. Conceptual Architecture



11. Important Point About Preview Features

Because this is a **preview feature**, code using it must be compiled and run with preview support for the corresponding JDK.

Conceptually:

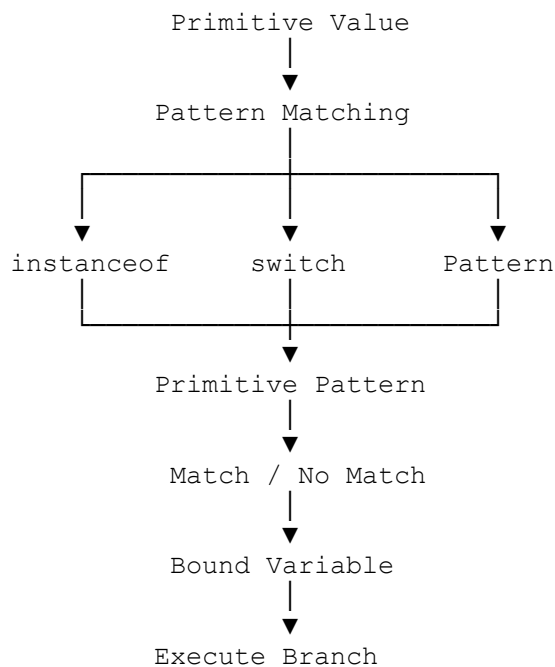
```
javac --enable-preview --release 27 PrimitiveTypesPatterns.java
```

and:

```
java --enable-preview PrimitiveTypesPatterns
```

The exact command-line requirements should be checked against the final JDK 27 release because preview APIs and language syntax can change before finalization.

12. Core Concept



Key takeaway

Primitive Types in Patterns, instanceof, and switch extends Java's pattern-matching model beyond reference types, allowing **primitive values such as int, long, float, and double to participate directly in pattern matching**. This makes numeric type testing, binding, and switch logic more uniform and expressive while reducing the need for manual conversions and branching.

Java 27 — PEM Encodings of Cryptographic Objects (Third Preview)

Java Version: 27

Year: 2026

Feature: PEM Encodings of Cryptographic Objects (Third Preview)

UniqueClassName: PEMEncodings

Status: Preview API. OpenJDK's Java 27 development documentation describes PEMEncoder, PEMDecoder, PEM, and the related cryptographic encoding APIs as preview functionality. PEM is a

textual format commonly used for cryptographic objects such as public/private keys, certificates, and CRLs. [OpenJDK](#)

1. Definition

PEM Encodings of Cryptographic Objects adds standardized Java APIs for encoding and decoding cryptographic material in **PEM (Privacy-Enhanced Mail)** format.

PEM represents binary cryptographic data as Base64 text surrounded by a type-specific header and footer. For example:

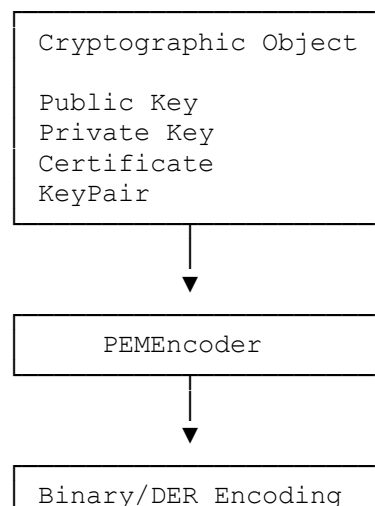
```
-----BEGIN PUBLIC KEY-----  
Base64EncodedData...  
-----END PUBLIC KEY-----
```

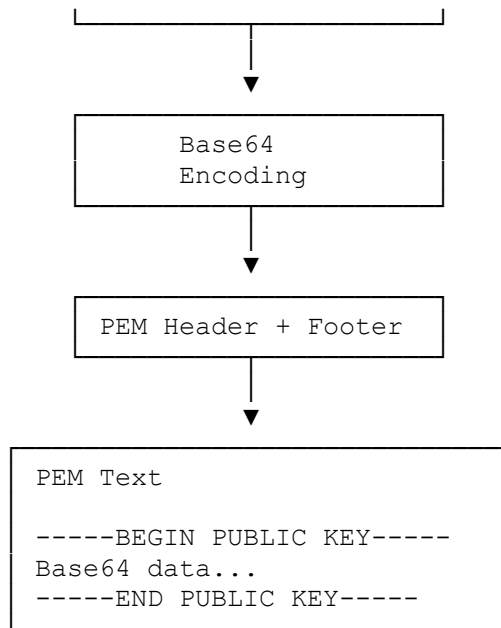
The Java API provides dedicated classes such as:

- PEMEncoder — converts supported cryptographic objects into PEM text.
- PEMDecoder — converts PEM text back into cryptographic objects.
- PEM — represents PEM data directly when there is no more specific cryptographic Java object.
- BinaryEncodable — common abstraction for objects that can be represented in standardized binary form. [OpenJDK](#)

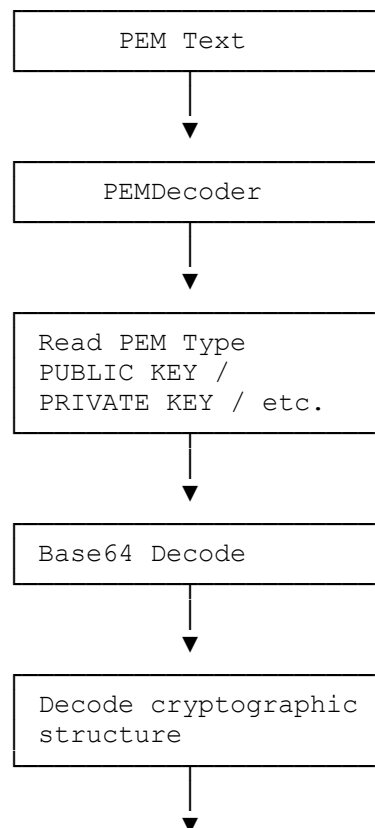
Supported objects include public/private keys, certificates, CRLs, encrypted private keys, and key pairs. [OpenJDK](#)

2. PEM Encoding Flowchart





3. Decoding Flowchart



Java Cryptographic Object

PEMDecoder can return an appropriate cryptographic object based on the PEM type, while the generic PEM representation can be used when the contents do not correspond to a supported Java cryptographic object. [OpenJDK](#)

4. Java Code Example — Generate and Encode a Key

The following example demonstrates the intended Java 27 API style using PEMEncoder.

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PEMEncoder;

public class PEMEncodings {

    public static void main(String[] args) throws Exception {

        // Generate an RSA key pair
        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("RSA");

        generator.initialize(2048);

        KeyPair keyPair =
            generator.generateKeyPair();

        // Create PEM encoder
        PEMEncoder encoder =
            PEMEncoder.of();

        // Encode the key pair as PEM
        String pem =
            encoder.encodeToString(keyPair);

        System.out.println(pem);
    }
}
```

PEMEncoder.encodeToString(...) returns the PEM representation as a String; the Java 27 API documentation specifies that a KeyPair is encoded using the PKCS #8 PRIVATE KEY representation. [OpenJDK](#)

Conceptual output

```
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBA...
...
-----END PRIVATE KEY-----
```

5. Encode a Public Key

You can also encode the public key separately:

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PEMEncoder;

public class PEMEncodings {

    public static void main(String[] args) throws Exception {

        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("RSA");

        generator.initialize(2048);

        KeyPair keyPair =
            generator.generateKeyPair();

        PEMEncoder encoder =
            PEMEncoder.of();

        String publicKeyPEM =
            encoder.encodeToString(
                keyPair.getPublic()
            );

        System.out.println(publicKeyPEM);
    }
}
```

Conceptual output:

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A...
...
-----END PUBLIC KEY-----
```

The Java 27 API identifies `PublicKey` as encoding to the PUBLIC KEY PEM type. [OpenIDK](#)

6. Encode an Encrypted Private Key

One particularly useful capability is password-protected private-key encoding.

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PEMEncoder;

public class PEMEncodings {

    public static void main(String[] args) throws Exception {

        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("RSA");

        generator.initialize(2048);

        KeyPair keyPair =
            generator.generateKeyPair();

        char[] password =
            "StrongPassword123!".toCharArray();

        PEMEncoder encoder =
            PEMEncoder.of()
                .withEncryption(password);

        String encryptedPEM =
            encoder.encodeToString(keyPair);

        System.out.println(encryptedPEM);
    }
}
```

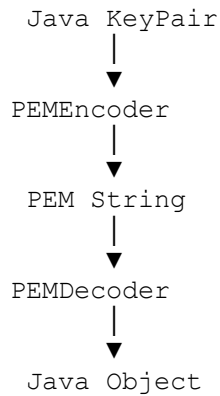
Conceptual output:

```
-----BEGIN ENCRYPTED PRIVATE KEY-----
MII...
...
-----END ENCRYPTED PRIVATE KEY-----
```

`withEncryption(char[])` returns an encoder configured to encrypt supported private-key material; the Java 27 API specifically lists `PrivateKey`, `KeyPair`, and `PKCS8EncodedKeySpec` for this configured encoder. [OpenJDK](#)

7. Encode → Decode

The overall process can be represented as:



Example:

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PEMDecoder;
import java.security.PEMEncoder;

public class PEMEncodings {

    public static void main(String[] args)
        throws Exception {

        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("RSA");

        generator.initialize(2048);

        KeyPair original =
            generator.generateKeyPair();

        // Encode
        PEMEncoder encoder =
            PEMEncoder.of();

        String pem =
            encoder.encodeToString(original);

        System.out.println("PEM:");
        System.out.println(pem);

        // Decode
        PEMDecoder decoder =
            PEMDecoder.of();

        KeyPair decoded =
            decoder.decode(
                pem,
```

```

        KeyPair.class
    );

    System.out.println(
        "Decoded algorithm: "
        + decoded.getPrivate()
        .getAlgorithm()
    );
}
}

```

The API is designed to make this kind of cryptographic-object conversion more direct instead of requiring application developers to manually implement PEM headers, Base64 processing, and format handling. [OpenIDK](#)

8. PEM Structure

A PEM object has three conceptual components:

-----BEGIN PUBLIC KEY-----
Base64-encoded content
-----END PUBLIC KEY-----

For a private key:

-----BEGIN PRIVATE KEY-----
Base64-encoded PKCS#8
-----END PRIVATE KEY-----

Common PEM type labels include:

```

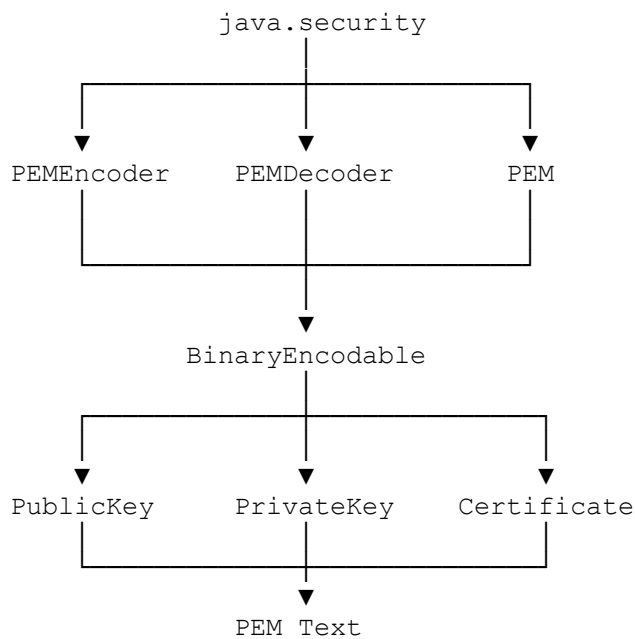
CERTIFICATE
PUBLIC KEY
PRIVATE KEY
ENCRYPTED PRIVATE KEY
X509 CRL
CERTIFICATE REQUEST

```

PKCS7
CMS

The PEM API documentation lists these as common type values. [OpenJDK](#)

9. Java API Architecture



The Java 27 API uses the common `BinaryEncodable` abstraction for cryptographic objects that can be represented in standardized binary form. [OpenJDK](#)

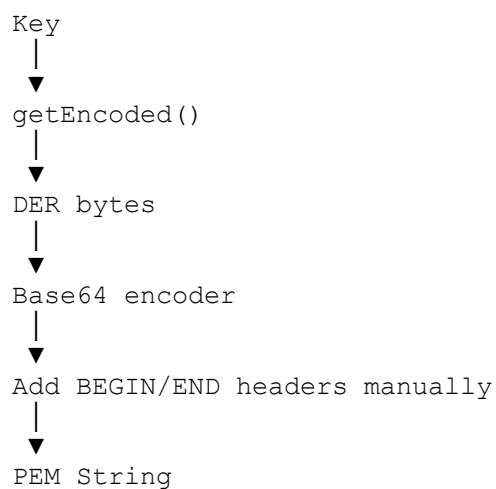
10. Advantages

Advantage	Description
Standard Java API	PEM handling becomes part of the JDK API
Less manual Base64 code	Encoding is handled by <code>PEMEncoder</code>
Cryptographic object support	Keys, certificates, CRLs, etc.

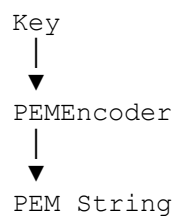
PEM decoding	PEM text can be converted back into Java objects
Encrypted private keys	Password-based private-key PEM encoding is supported
Type-aware processing	PEM labels determine the expected cryptographic object
Interoperability	PEM is widely used by TLS, OpenSSL, PKI, and security tooling
Cleaner security code	Less custom parsing and encoding logic

11. Before vs Java 27 API

Traditional approach



Java 27 approach



This is the major usability improvement.

12. Preview Compilation

Because this is a **preview API**, preview support is required when using the corresponding JDK 27 build.

Conceptually:

```
javac --enable-preview --release 27 PEMEncodings.java
```

and:

```
java --enable-preview PEMEncodings
```

The exact API may still evolve because preview features are explicitly subject to change before becoming permanent. [OpenJDK](#)

Key takeaway

PEM Encodings of Cryptographic Objects brings native PEM encoding and decoding capabilities into the Java security APIs. Instead of manually converting keys and certificates to DER, Base64-encoding them, and adding PEM headers and footers, developers can use **PEMEncoder** and **PEMDecoder** to work directly with standardized PEM representations. [OpenJDK](#)